

Round-Optimal Privacy Preserving Authenticated Key Exchange Even for Incomplete Sessions

Xavier Bultel^[0000–0002–8309–8984] and Khouredia Cisse^[0009–0001–7203–7756]

INSA Centre Val de Loire, Université d'Orléans, Inria, France
{xavier.bultel,khouredia.cisse}@insa-cvl.fr

Abstract. Several modern applications, such as Signal or WireGuard, use efficient Noise-like *implicit* authentication key exchanges that require only a small number of exponentiations and two interactions. These protocols have been proven to be secure under the 'strong Diffie–Hellman' (SDH) assumption in the random oracle model (ROM). At ESORICS 2021, Ramacher, Slamanig and Weninger presented an extension to the implicit authenticated key exchange security model, which enables strong privacy properties to be captured in addition to key-secrecy, including man-in-the-middle privacy (ensuring privacy even if a session is interrupted) and forward privacy. They also proposed a protocol to instantiate their model. In this paper, we present an efficient Noise-like protocol that achieve privacy in this model. Our protocol is as efficient as Noise-like protocols that do not guarantee privacy in terms of exponentiations. Moreover, our protocol requires three interactions, which is optimal for this privacy model. It is also more efficient than the ESORICS 2021 protocol in terms of both exponentiations and interactions. Finally, we propose another round-optimal protocol, slightly less efficient in terms of exponentiations, but secure under the CDH assumption in the ROM.

Keywords: Authenticated Key Exchange · Privacy · Provable security.

1 Introduction

End-to-end encryption and key exchange protocols are among the most fundamental tools in cryptography and have been widely used in practice for several decades. These protocols must be resistant to man-in-the-middle attacks, whereby an active attacker can alter communications. A standard method involves running a Diffie–Hellman key exchange, whereby Alice sends an ephemeral key $X = g^x$ and Bob sends $Y = g^y$, and the shared secret key is $K = g^{xy}$. Alice and Bob are also equipped with long-term signature keys generated a priori, which allows them to sign the transcript of the key exchange (i.e. the pair (X, Y)). As Bob already knows X when he sends Y , he can send his signature at the same time. However, Alice must send her signature after receiving Y . This method therefore requires three interactions. If any of these interactions are altered, Alice and Bob can detect this and abort the protocol, making active

attacks ineffective. Under the impetus of the Noise framework¹, more efficient *implicit* authenticated key exchange protocols have been deployed in several real-world applications (e.g. Signal [11] or WireGuard [7]). In these protocols, authentication is implicit in the sense that users no longer sign the transcript, saving the last interaction, but instead use their long-term secret keys, in addition to g^{xy} , to generate the shared key. Therefore, even if an adversary can alter the interactions, they will not be able to share keys with Alice and Bob, meaning the attack will be ineffective for decrypting subsequent messages.

In [6], Cohn-Gordon *et al.* propose an optimised version of this type of protocol (and provide proof of optimal security in terms of reduction loss): Alice and Bob have long-term public keys $U = g^u$ and $V = g^v$, they exchange $X = g^x$ and $Y = g^y$, then compute the shared secret key $K = H(U\|V\|X\|Y\|g^{ux}\|g^{vy}\|g^{xy})$, where H is a hash function modeled as a random oracle. Knowledge of the long-term keys is necessary to compute Y^u for Alice and X^v for Bob, and knowledge of the ephemeral keys is necessary to compute, conversely, U^x for Alice and V^y for Bob, which prevents the adversary from computing a key shared with Alice by impersonating Bob, and vice versa. Forward secrecy is ensured by g^{xy} ; if the adversary finds the long-term keys a posteriori, the element g^{xy} does not depend on them, which prevents them from guessing K . Note that in this work, the authors implicitly assume that the responder, Bob, knows a priori that his interlocutor is Alice before receiving the first message containing X (otherwise he would not be able to compute U^y). In practice, Alice would have to send her identity to Bob in clear text before the protocol (since Alice and Bob do not have a shared key a priori). As a result, by its very nature, this protocol does not protect user identity.

In [13], Ramacher *et al.* propose to extend the model given by Cohn-Gordon *et al.* in order to capture two additional strong privacy properties: man-in-the-middle (MITM) privacy and forward privacy. However, this time the initiator must explicitly send their identity to the responder during the protocol, to prevent an adversary from deducing it. In their setup, Alice (the initiator) knows the long-term public key of the intended recipient (Bob). However, the responder does not initially know the identity of the party contacting them and only learns it during protocol execution. In MITM privacy, they consider an active adversary whose goal is to guess the identity of one of the two parties whose long-term keys are not corrupted during the execution of the protocol. Note that this model differs from other similar models in the literature in that it captures privacy even for incomplete sessions. In forward privacy, they consider an adversary attempting to guess the identity of the parties after the protocol has been successfully executed. The attacker is therefore not active during the interaction, but has access to the transcript and can corrupt the long-term keys of any user after the fact. The authors demonstrate that neither of these two privacy properties implies the other, and provide a protocol that verifies key-secrecy, MITM privacy, and forward privacy.

¹ See <http://www.noiseprotocol.org/>.

In this article, we propose efficient protocols (in terms of the number of exponentiations and interactions) that verify key-secrecy, MITM privacy, and forward privacy in the models of [6, 13]. We propose a first protocol based on the strong Diffie-Hellman assumption, which is similar to the computational Diffie-Hellman assumption (guessing g^{xy} from $X = g^x$ and $Y = g^y$ in a prime order group is difficult), except that the adversary also has an oracle $\text{stDH}_x(A, Z)$ allowing them to test whether $Z = A^x$. Note that this assumption is required for the protocol proposed in [6]. Our protocol is more efficient than that of [13] and is also optimal in terms of the number of interactions. In terms of exponentiations, our protocol is similar to [6] (which does not have privacy). We also propose a variant of our protocol using Schnorr's signatures of knowledge that can be proven secure and private under the CDH assumption, which is weaker than SDH and is considered more standard. However, this protocol is slightly less computationally efficient, as it requires a few additional exponentiations.

1.1 Technical Overview of our Contributions

Our SDH-based protocol Π_{SDH} is shown in Figure 1, where g is the generator of a group of prime order p , H is a hash function modeled as a random oracle, Enc/Dec are the encryption/decryption algorithms of an authenticated symmetric encryption scheme, and ctxt denotes the transcript (X, Y, C) of the protocol. This protocol is similar to Noise, except that Alice and Bob compute the key $K \leftarrow H(g^{vx} \| g^{yx} \| X \| Y)$, then Alice uses it to encrypt her public key (associated with her identity) and sends the ciphertext to Bob. Bob discovers Alice's identity by decrypting this public key (remember that before this, Bob does not know who he is communicating with). After verifying that the public key corresponds to a registered user (in practice, Bob verifies that the public key is certificated), Bob generates the shared key in an implicit authentication manner.

Alice (initiator)	Bob (responder)
$\text{sk}_i = u; \text{pk}_i = U = g^u$	$\text{sk}_r = v; \text{pk}_r = V = g^v$
$x \leftarrow \mathbb{Z}_p; X \leftarrow g^x$	$\xrightarrow{X} y \leftarrow \mathbb{Z}_p; Y \leftarrow g^y$
$h_1 \leftarrow (V^x \ Y^x)$	$\xleftarrow{Y} h_1 \leftarrow (X^v \ X^y)$
$K \leftarrow H(h_1 \ X \ Y)$	$K \leftarrow H(h_1 \ X \ Y)$
$C \leftarrow \text{Enc}_K(U)$	$\xrightarrow{C} U \leftarrow \text{Dec}_K(C)$
	If U is not a registered public key, Bob aborts the protocol.
$h_2 \leftarrow Y^u$	$h_2 \leftarrow U^y$
$K_i \leftarrow H(\text{ctxt} \ h_1 \ h_2)$	$K_r \leftarrow H(\text{ctxt} \ h_1 \ h_2)$

Fig. 1. Private authenticated key exchange protocol based on SDH (Π_{SDH}).

Key-secrecy can be proved in a similar way to [6]. Remember that an active adversary cannot re-compute the final keys K_i and K_r , so they cannot use them to determine Alice’s or Bob’s identity. To attack privacy, the ciphertext containing Alice’s public key must therefore be exploited. As long as Bob remains uncorrupted, an adversary who intercepts messages sent by Alice cannot deduce $g^{xv} = V^x = X^v$, and therefore cannot deduce any information about the key K , which appears to be random. Thus, disguising Alice’s identity amounts to attacking the secure encryption, which ensures MITM privacy. On the other hand, for forward privacy, if the adversary accesses the transcript after the key exchange, they are unable to compute $X^y = Y^x = g^{xy}$ even if they know the long-term keys u and v . Once again, the key K seems random to them, and guessing Alice’s identity is equivalent to attacking the encryption. In this protocol, each user performs four exponentiations, and the number of interactions is optimal for forward privacy: Alice cannot send her identity during the first interaction because she does not share any secrets with Bob. This would mean that Bob could discover Alice’s identity solely using his long-term key, which is impossible if the protocol is forward private. There are therefore at least 3 interactions.

Furthermore, by adding Schnorr’s Signature Of Knowledge (SOK) [16] of discrete logarithm on ephemeral keys (signing the user role) and public keys (signing the user identity), we construct a slightly less efficient protocol (three additional exponentiations per user during a session, and two additional exponentiations when verifying the public key certificate to also verify the SOK), but that can be proven under the CDH assumption instead of SDH. SDH allows, during a reduction, to artificially verify via an oracle whether, for a given CDH share $X = g^x$, the triplet (X, A, Z) verifies $Z = A^x$ or not for a chosen (A, Z) . Our technique is to attach an extractable SOK to each element $A = g^a$, allowing a to be extracted to verify whether $Z = X^a$ instead.

In Figure 2 we compare the efficiency, assumptions, and security of our protocols with those of [6, 13]. For [13], we only consider the protocol achieving both (strong) MITM and forward privacy, which we instantiate with Diffie-Hellman key exchange, ElGamal encryption [9] and Schnorr signature [16]. We observe that our Π_{SDH} is more efficient in terms of both interactions and exponentiations than the only proven (strongly) private protocol in the literature [13].

1.2 Related work

As mentioned previously, our work is based on the privacy-preserving authenticated key exchange (PPAKE) model proposed by [13], but it should be noted that there are other competing PPAKE models. The earliest attempts to (informally) capture privacy in key agreement can be traced back to Aiello et al. [2], who introduced the idea of keeping a participant’s identity hidden from unauthorised parties, including active attackers impersonating peers. Although they proposed two specific protocols, one for protecting the initiator’s identity and the other for protecting the responder’s, their notion of privacy was neither formally defined nor analysed. Prior to this, Canetti and Krawczyk [4] briefly mentioned

Protocol	Assump.	Privacy	Inter.	Exp. (Alice)	Exp. (Bob)
Π [6]	SDH (ROM)	$\times$	2	4	4
Π_{Twin} [6]	CDH (ROM)	$\times$	2	8	7
Π_{Com} [6]	DDH (ROM)	$\times$	4	6	6
$\Pi_{\text{PKE}}^{\dagger}$ [13]	Generic (ROM)	$\checkmark$	4	5	5
Π_{SDH}	SDH (ROM)	$\checkmark$	3	4	4
Π_{CDH}	CDH (ROM)	$\checkmark$	3	7*	7*

Fig. 2. Comparison of our work with [6] and [13]. * Π_{CDH} also requires an additional exponentiation when generating long-term keys, and two additional exponentiations when verifying a certificate.

the concept of identity concealment through encryption, albeit without providing a formal framework. Zhao [19] introduced the CAKE model, which formalises forward identity privacy, and limited MITM privacy. However, in this model, the MITM-privacy is only guaranteed if the key exchange is successful, which seems insufficient in practice, because even if the adversary who disrupted the exchange is unable to read the communication, they still learn the identities of the users who wish to communicate. To the best of our knowledge, comparing to prior models [15, 19] only the one proposed by [13] considers MITM attacks on incomplete sessions.

Another line of work involves studying how to add privacy to real-world protocols. Schäge et. al. [15] proposed a tailored PPAKE model and applied it to real-world protocols such as IKEv2. However, their model also assumes prior knowledge of participants’ identities and targets specific deployment contexts. On the other hand, the latest versions of TLS have been adapted to ensure certain (weak) privacy properties, which have been analysed in [3]. However, as the general structure of TLS has not been modified, it does not guarantee properties as strong as those of [15].

In a more recent article, Lyu et al. [10] propose another PPAKE model that differs from [13], and provide a protocol proven in the standard model requiring only 3 exchanges. The authors claim that their model is equivalent to the one in [13], and claim an attack on the forward-privacy of the protocol in [13]. However, as added in the ‘follow-up work’ section of the technical report [14] of [13], the two models differ because [10] assumes that messages are sent on broadcasting channels and that every potential receiver answers to every received message. Furthermore, the attack does not hold in the forward-privacy model of [13], since it considers an active attacker during the session, whereas [13] considers passive attackers only who can corrupt the long-term keys a posteriori. Furthermore, we remark that the protocol in [10] does not achieve the MITM-privacy definition given in [13]. In the MITM-privacy of [13], a user only reveals their identity to another user during a session if they are certain that they are communicating with the right person (formally, the attacker must guess the identity of a user in a session where they either do not yet know the identity of their interlocutor, or they know it and it is an uncorrupted user). In the protocol in [10], the first

flow consists of Alice sending a MAC key to Bob encrypted with Bob's long-term key, then Bob uses this key to generate a MAC on they response. Thus, an adversary can verify whether a user is Bob or not by simply sending the first flow of the protocol, because only Bob will be able to respond, and then interrupt the session. In essence, once again, this paper does not consider MITM-privacy for interrupted sessions.

2 Preliminaries

In this section, we recall some cryptographic tools.

Definition 1 (Computational Diffie-Hellman assumptions). *Let λ be a security parameter, and let $\mathbb{G}$ be a cyclic group of prime order $p > 2^\lambda$ with generator g . Let $x, y \xleftarrow{\$} \mathbb{Z}_p$ be two scalars sampled uniformly at random.*

- The **Computational Diffie-Hellman** (CDH) assumption in $\mathbb{G}$ states that for any Probabilistic Polynomial-Time (PPT) in λ algorithm $\mathcal{A}$, the advantage $\text{Adv}_{\mathcal{A}}^{\text{CDH}}(\lambda) = \Pr[\mathcal{A}(g^x, g^y) = g^{xy}]$ of $\mathcal{A}$ over CDH is negligible.
- The **Strong Diffie-Hellman** (SDH) assumption in $\mathbb{G}$ is defined as CDH except that $\mathcal{A}$ can access a test oracle $\text{stDH}_x(A, Z)$ that returns 1 if $Z = A^x$, 0 otherwise: it states that for any PPT algorithm $\mathcal{A}$, the advantage $\text{Adv}_{\mathcal{A}}^{\text{SDH}}(\lambda) = \Pr[\mathcal{A}^{\text{stDH}_x(\cdot, \cdot)}(g^x, g^y) = g^{xy}]$ of $\mathcal{A}$ over SDH is negligible. Note that, equivalently, the test oracle can be stDH_y , but the adversary cannot have access to both stDH_x and stDH_y .

Theorem 7 in [1] proves that SDH holds in Shoup's generic group model. SDH trivially implies CDH.

Definition 2. (Authenticated Encryption [12]) *Let λ be a security parameter. A Symmetric Encryption scheme SE is a triplet $(\mathcal{K}, \text{Enc}, \text{Dec})$, where $\mathcal{K}$ is a secret key set verifying $|\mathcal{K}| > 2^\lambda$, $\text{Enc}_k(m)$ is an encryption algorithm that takes as input a secret key $k \in \mathcal{K}$ and a message m , and returns a ciphertext c , and $\text{Dec}_k(c)$ is a decryption algorithm that takes as input a secret key $k \in \mathcal{K}$ and a ciphertext c , and returns a message m .*

Moreover, we define the authenticated encryption (AE) experiment $\text{Exp}_{\text{SE}, b, \mathcal{A}}^{\text{AE}}(\lambda)$ for the two-stage adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ against SE as follows, where $\text{Enc}_k(\cdot)$ and $\text{Dec}_k(\cdot)$ are encryption/decryption oracles such that $\text{Dec}_k(\cdot)$ returns $\perp$ on c , and returns $\perp$ on any query when $b = 1$:

$\text{Exp}_{\text{SE}, b, \mathcal{A}}^{\text{AE}}(\lambda) :$
 $k \xleftarrow{\$} \mathcal{K}; b \xleftarrow{\$} \{0, 1\}$
 $(m_0, m_1) \leftarrow \mathcal{A}_1^{\text{Enc}_k(\cdot), \text{Dec}_k(\cdot)}(\lambda)$
 $c \leftarrow \text{Enc}_k(m_b)$
 returns $b' \leftarrow \mathcal{A}_2^{\text{Enc}_k(\cdot), \text{Dec}_k(\cdot)}(c)$

SE is said to be AE secure if for any PPT two-stage adversary $\mathcal{A}$, the advantage $\text{Adv}_{\text{SE}, \mathcal{A}}^{\text{AE}}(\lambda) = \left| \Pr[\text{Exp}_{\text{SE}, 0, \mathcal{A}}^{\text{AE}}(\lambda) = 1] - \Pr[\text{Exp}_{\text{SE}, 1, \mathcal{A}}^{\text{AE}}(\lambda) = 1] \right|$ is negligible.

For any of these assumptions/properties denoted X , we define the advantage over X as $\text{Adv}^X(\lambda) = \max_{\mathcal{A} \in \text{Poly}(\lambda)} \text{Adv}_{\mathcal{A}}^X(\lambda)$, where $\text{Poly}(\lambda)$ refers to the set of all the probabilistic polynomial-time (possibly two-stage) algorithms in λ .

For any positive integer x , we write $[x] = \{1, 2, \dots, x\}$ for the set of the first x positive integers.

3 Security Model

We rely on the authenticated key exchange (AKE) model of [6] for key-secrecy, and the extension of [13] for MITM and forward privacy. Note that in [13], a weak MITM-privacy is also proposed, which assumes that no user is ever corrupted. Since we do not use this weak property, we do not mention it here.

Execution environment. We consider μ parties. Each party P_i is represented by a set of oracles $\{\pi_i^1, \dots, \pi_i^\ell\}$, where each oracle corresponds to a session (a single execution of a protocol role), and where $\ell \in \mathbb{N}$ is the maximum number of protocol sessions per party. Each oracle π_i^s has access to the long-term key pair $(\text{sk}_i, \text{pk}_i)$ of the party P_i and the list L_{pk} containing the indices/public keys (i, pk_i) of all the parties. It also maintains a list of internal state variables that are described in the following. For each oracle π_i^s , the corresponding variables are initially set to the empty string $\perp$.

- Pid_i^s ("peer id") stores the identity of the intended communication partner. Note that in private key exchange protocols, a user may not know the identity of their partner at the start of a session. In this case, this variable will remain set to $\perp$ until the partner is identified.
- $\Psi_i^s \in \{\perp, \text{Accept}, \text{Reject}\}$ indicates if oracle π_i^s has successfully completed the protocol execution and "accepted" the resulting key, or aborted the protocol and "rejected" the key exchange. Until the protocol is complete or aborted, this value remains $\perp$. As a prerequisite, we stress that Pid_i^s must be instantiated (i.e. $\text{Pid}_i^s \neq \perp$) for Ψ_i^s to be set to **Accept**.
- k_i^s stores the session key computed by π_i^s . Until the protocol is complete, this value remains $\perp$. It is assigned only if the oracle π_i^s reaches the **Accept** state. In other words: $k_i^s \neq \perp \Leftrightarrow \Psi_i^s = \text{accept}$.
- $\text{esk}_i^s/\text{epk}_i^s$: stores P_i 's ephemeral secret/public keys specified by the protocol. These keys are generated specifically for each session by the parties involved and are renewed for each new session.
- $\overline{\text{epk}}_i^s$: stores the ephemeral public key of the P_i 's partner. This key is specific to the session and is received during an interaction specified in the protocol. Note that this key can be received before the partner's identity is learned by the oracle.
- $\text{role}_i^s \in \{\perp, \text{initiator}, \text{responder}\}$ indicates π_i^s 's role during the protocol.

Additionally, the environment keeps track of three initially empty lists: L_{corr} , containing all corrupted parties, L_{Send} , storing all sent messages, and L_{SessKey} , recording all established session keys.

Oracles and attacker model. Our attacker $\mathcal{A}$ has a full control over the communication network. Furthermore, the attacker knows the list of parties and their respective public keys. It interacts with the oracles through queries.

- **Send**(i, s, m): This allows $\mathcal{A}$ to send an arbitrary message m to oracle π_i^s if it exists. The oracle will respond according to the protocol specification and its current internal state. To start a new oracle, the message m takes a special form (**START** : role, j). If π_i^s was already initialized before, **Send** returns $\perp$. Otherwise, the oracle initializes π_i^s in the role **role** by setting $\text{role}_i^s = \text{role}$. If **role** = **initiator**, then it sets $\text{Pid}_i^s = j$ and outputs the first message of the protocol. Else, if **role** = **responder**, the partner identity remains $\text{Pid}_i^s = \perp$ (In this case, j is not used and can be set to the empty string $\perp$ in the query). Note that, in the case of the responder, the oracle does not receive the first message of the protocol during initialisation (it will receive it in the second query).
- **RevLTK**(i): For $i \leq \mu$, this returns the long-term private key sk_i of party P_i . After this query, P_i and all its oracles π_i^s (for any s) are said to be corrupted.
- **RegisterLTK**(i, pk_i): For $i > \mu$, this allows the adversary to register a new party P_i with the public key pk_i . The adversary is not required to know the corresponding private key. After this query, the pair (i, pk_i) is added to L_{pk} . Parties registered by **RegisterLTK**(i, pk_i) (and their oracles) are corrupted by definition.
- **RevSessKeySec**(i, s): This allows the adversary to learn the session key derived by an oracle. If $\Psi_i^s = \text{accept}$, return k_i^s . Otherwise, return $\perp$. After this query, π_i^s is said to be revealed.

Furthermore, privacy properties require an alternative definition of the session key revelation oracle, which uses the definition of *matching conversations*.

Definition 3 (Matching Conversation). *Let Π be an N -message two-party protocol in which all messages are sent sequentially. If a session oracle π_i^s sent (resp. receive) the last message of the protocol, then π_j^t is said to have matching conversations to π_i^s if the first $N - 1$ (resp. all N) messages of π_i^s transcript agrees with the first $N - 1$ (resp. all) messages of π_j^t 's transcript (excluding the initialisation messages received by both oracles).*

- **RevSessKeyPriv**(i, s): This is an alternative oracle to learn the session key. If there already is an entry (π_i^s, k^*) in L_{SessKey} , or if there is an entry (π_j^t, k^*) so that π_i^s and π_j^t have matching conversation, this oracle returns k^* . If $\Psi_i^s = \text{accept}$, it sets $k^* \leftarrow k_i^s$, adds (π_i^s, k^*) to L_{SessKey} , and returns k^* , else if $\Psi_i^s = \text{reject}$, it picks a random session key k^* , adds (π_i^s, k^*) to L_{SessKey} , and returns k^* . Otherwise (i.e. If $\Psi_i^s = \perp$), it returns $\perp$. After this query, π_i^s is said to be revealed.

Partnering. Here we recall the definitions used to formally determine whether a session key is corrupted or not, and whether two session oracles correspond to the execution of a protocol between the two corresponding parties.

Definition 4 (Origin-oracle). An oracle π_j^t is an origin-oracle for an oracle π_i^s if $\Psi_j^t \neq \perp$, $\Psi_i^s = \text{Accept}$ and the messages sent by π_j^t equal the messages received by π_i^s , except for the initialisation message received by π_i^s .

Definition 5 (Partner oracles). We say that two oracles π_i^s and π_j^t are partners if (i) each is an origin-oracle for the other; (ii) each one's identity is the other one's peer identity, i.e., $\text{Pid}_i^s = j$ and $\text{Pid}_j^t = i$; and (iii) they do not have the same role, i.e., $\text{role}_i^s \neq \text{role}_j^t$, $\text{role}_i^s \neq \perp$, and $\text{role}_j^t \neq \perp$.

Security experiment. The security model is formally defined as a game between an adversary, denoted by $\mathcal{A}$, and a challenger, denoted by $\mathcal{C}$. In this game, the adversary may issue the standard set of queries described earlier. Additionally, they are granted access to a special oracle $\text{Test}(m)$ depending on a secret random bit b that the adversary has to guess in order to win. In key-secrecy, the oracle returns either the key for a given session or a random key. For privacy, this oracle initializes a session for one of two parties chosen by the adversary. Since our main focus is privacy, we defer the key-secrecy experiment in Appendix A.

Privacy. The oracle $\text{Test}(m)$ is defined as follows: for $m = (X, i, j)$ such that $X \in \{\text{MITM-privacy}, \text{Forward-privacy}\}$ and $i, j \leq \mu$, it creates a new party $P_{i|j}$ with identifier $i|j$. This party has all the properties of P_i (if $b = 0$) or P_j (if $b = 1$), but no active sessions. The public key of $P_{i|j}$ is not announced to the adversary and the query $\text{RevLTK}(i|j)$ always returns $\perp$. Furthermore it creates exactly one session $\pi_{i|j}^1$, and returns the new identifier $i|j$.

The experiment $\text{Exp}_{\text{PPAKE}, \mathcal{A}}^X(\lambda)$, with X the label sent to the oracle Test , is defined as follows:

1. Let μ be the number of parties in the game and ℓ the number of sessions per user. $\mathcal{C}$ begins by drawing a random bit $b \xleftarrow{\$} \{0, 1\}$ and generating key pairs $\{(\text{sk}_i, \text{pk}_i) \mid 1 \leq i \leq \mu\}$ as well as oracles $\{\pi_i^s \mid 1 \leq i \leq \mu, 1 \leq s \leq \ell\}$.
2. $\mathcal{C}$ now runs $\mathcal{A}$, providing all the public keys as input. During its execution, $\mathcal{A}$ may adaptively issue $\text{Send}(i, s, m)$, $\text{RevLTK}(i)$, $\text{RevSessKeyPriv}(i, s)$ and $\text{RegisterLTK}(i, \text{pk}_i)$ queries any number of times and the $\text{Test}(m)$ query once.
3. Depending on what argument X the $\text{Test}(m)$ oracle was called with, we require the corresponding property below to hold through the entire game.
 - (a) **MITM-privacy:** P_i and P_j are never corrupted. Furthermore, we require that $\text{Pid}_{i|j}^1 = \perp$ or $\text{Pid}_{i|j}^1 = k$ for some $k \leq \mu$, while P_k is never corrupted.
 - (b) **Forward-privacy:** The returned oracle $\pi_{i|j}^1$ has a partner oracle π_k^r at the end of the game. Furthermore, no oracle besides π_k^r may be instructed to start a protocol run with intended partner $P_{i|j}$.
4. The game ends when $\mathcal{A}$ terminates with the output b' representing the guess of the secret bit b . If $b' = b$, it outputs 1. Otherwise, it outputs 0.

Definition 6. Let PPAKE be a key exchange protocol. The advantage of an adversary $\mathcal{A}$ in breaking the $X \in \{\text{MITM-privacy}, \text{Forward-privacy}\}$ security of protocol PPAKE is defined as:

$$\text{Adv}_{\text{PPAKE}, \mathcal{A}}^X(\lambda) = \left| \Pr \left[\text{Exp}_{\text{PPAKE}, \mathcal{A}}^X(\lambda) = 1 \right] - \frac{1}{2} \right|.$$

4 Protocol Π_{SDH}

In this section, we prove that the protocol Π_{SDH} in Figure 1, already presented informally in the introduction, has the three properties of our security model under the SDH assumption in the ROM.

In our proofs, during a session simulated by the oracle π_i^s where P_i is the initiator, $(\text{pk}_i, \text{sk}_i)$ corresponds to (U, u) , $(\text{pk}_{\text{Pid}_i^s}, \text{sk}_{\text{Pid}_i^s})$ corresponds to (V, v) and $(\text{epk}_i^s, \text{esk}_i^s)$ corresponds to (X, x) in Figure 1. When P_i is the responder, $(\text{pk}_i, \text{sk}_i)$ corresponds to (V, v) , $(\text{epk}_i^s, \text{esk}_i^s)$ corresponds to (Y, y) , and $(\text{pk}_{\text{Pid}_i^s}, \text{sk}_{\text{Pid}_i^s})$ corresponds to (U, u) when defined. Recall that when the oracle π_i^s is initialised for the role **responder**, the value of Pid_i^s is set to $\perp$. In accordance with the protocol in Figure 1, Pid_i^s is updated after the third query is received on the condition that the message received is well-formed, i.e., its decryption gives the public key of a registered user P_j . In this case, Pid_i^s will be set to j .

Theorem 1. *Our protocol Π_{SDH} instantiated with a secure AE scheme is MITM-private under the SDH assumption in the ROM.*

We prove MITM-privacy in two steps. First, we show that the adversary can never guess and send the Diffie-Hellman values of V^x and U^y to the RO instead of uncorrupted users (owners of V and U) under the SDH assumption in the random oracle. This result (Lemma 1) allows us to show that during the MITM-privacy experiment, the key K used by $P_{i||j}$ is perfectly random, and therefore that the identity encrypted in C can only be recovered by attacking the secure authenticated encryption scheme.

Lemma 1. *Let E be the event: "at the end of the MITM-privacy experiment, for some tuple (s, i, j, R) , P_i and P_j are never corrupted, $\text{Pid}_i^s = j$, and the R^{th} query to the RO contains the value $g^{\text{esk}_i^s \text{sk}_j}$, where esk_i^s is the ephemeral key used in π_i^s ". For any PPT adversary $\mathcal{A}$ playing the MITM-privacy experiment on Π_{SDH} , the probability that E occurs and $\mathcal{A}$ wins the experiment is negligible under the SDH assumption in the ROM.*

Proof. We show that $\Pr \left[\text{Exp}_{\Pi_{\text{SDH}}, \mathcal{A}}^{\text{MITM-privacy}}(\lambda) = 1 \wedge E \right]$ is negligible using the following sequence of games.

Game G_0 : This is the experiment $\text{Exp}_{\Pi_{\text{SDH}}, \mathcal{A}}^{\text{MITM-privacy}}(\lambda)$ except that if E does not occur, the experiment returns 0. We have that:

$$\Pr \left[\text{Exp}_{\Pi_{\text{SDH}}, \mathcal{A}}^{\text{MITM-privacy}}(\lambda) = 1 \wedge E \right] = \Pr [G_0 \text{ returns } 1].$$

Game G_1 : G_1 is the same as G_0 except that the challenger picks $(s_*, i_*, j_*) \xleftarrow{\$} [\ell] \times [\mu]^2$, and returns 0 if E does not occur on the tuple (s_*, i_*, j_*, R_*) for some query index R_* . We have that:

$$\Pr [G_0 \text{ returns } 1] \leq \ell \mu^2 \Pr [G_1 \text{ returns } 1].$$

We will show that $\Pr[G_1 \text{ returns } 1] \leq \text{Adv}^{\text{SDH}}(\lambda)$.

Let $\mathcal{A} \in \text{Poly}(\lambda)$ be an adversary against G_1 . We build the following algorithm $\mathcal{B} \in \text{Poly}(\lambda)$ against SDH. $\mathcal{B}(\mathbf{A}, \mathbf{B})$ perfectly simulates G_1 to $\mathcal{A}$, except that:

- If at some stage $\text{Pid}_{i_*}^{s_*} \neq \perp$ and $\text{Pid}_{i_*}^{s_*} \neq j_*$, $\mathcal{B}$ aborts the game (G_1 will return 0 anyway).
- $\mathcal{B}$ sets $\text{pk}_{j_*} \leftarrow B$, and sets $\text{epk}_{i_*}^{s_*} \leftarrow A$, where $\text{epk}_{i_*}^{s_*}$ is the ephemeral public key used in $\pi_{i_*}^{s_*}$, noted X in the protocol (resp. Y) for the initiator (resp. responder). Note that $\mathcal{B}$ does not know the respective discrete logarithms sk_{j_*} and $\text{esk}_{i_*}^{s_*}$ of pk_{j_*} and $\text{epk}_{i_*}^{s_*}$. It is therefore necessary to specify how $\mathcal{B}$ simulates the oracles that would use these values, i.e. oracles $\pi_{j_*}^s$ for all s for sk_{j_*} and the oracle $\pi_{i_*}^{s_*}$ for $\text{esk}_{i_*}^{s_*}$. These two cases are detailed in the two following items.
- **For simulating the oracles $\pi_{j_*}^s$ for all s** , each time that $\mathcal{B}$ should compute $(\overline{\text{epk}_{j_*}^s})^{\text{sk}_{j_*}}$, it sets a tuple $T_s \leftarrow (\overline{\text{epk}_{j_*}^s}, \text{pk}_{j_*})$. If, afterward, $\mathcal{B}$ needs to compute the hash of an element containing $(\overline{\text{epk}_{j_*}^s})^{\text{sk}_{j_*}}$ using the random oracle, it replaces $(\overline{\text{epk}_{j_*}^s})^{\text{sk}_{j_*}}$ with T_s and labels the query as 'incomplete' when storing it in the list of random oracle queries (but still returns a random value corresponding to the query).
- **For simulating the oracles $\pi_{i_*}^{s_*}$** , we distinguish two cases:

There exists $(i, s) \neq (i_*, s_*)$ such that $\text{epk}_i^s = \overline{\text{epk}_{i_*}^{s_*}}$: In this case, $\mathcal{B}$ knows the secret key esk_i^s . Each time that $\mathcal{B}$ should compute $(\overline{\text{epk}_{i_*}^{s_*}})^{\text{esk}_{i_*}^{s_*}}$, it computes $(\text{epk}_{i_*}^{s_*})^{\text{esk}_i^s}$ instead. Each time that $\mathcal{B}$ should compute $\text{pk}_{j_*}^{\text{esk}_{i_*}^{s_*}}$, it sets the tuple $T \leftarrow (\text{epk}_{i_*}^{s_*}, \text{pk}_{j_*})$. If, afterward, $\mathcal{B}$ needs to compute the hash of an element containing $\text{pk}_{j_*}^{\text{esk}_{i_*}^{s_*}}$ using the random oracle, it replaces $\text{pk}_{j_*}^{\text{esk}_{i_*}^{s_*}}$ by T , and labels the query as 'incomplete' when storing it in the list of random oracle queries (but still returns a random value corresponding to the query).

There is no $(i, s) \neq (i_*, s_*)$ such that $\text{epk}_i^s = \overline{\text{epk}_{i_*}^{s_*}}$: In this case, $\mathcal{B}$ does not know the discrete logarithm of $\overline{\text{epk}_{i_*}^{s_*}}$. We distinguish two cases:

- If $\text{role}_{i_*}^{s_*} = \text{initiator}$, each time that $\mathcal{B}$ should compute $(\overline{\text{epk}_{i_*}^{s_*}})^{\text{esk}_{i_*}^{s_*}}$, it sets the result as $\perp$ instead. Each time that $\mathcal{B}$ should compute $\text{pk}_{j_*}^{\text{esk}_{i_*}^{s_*}}$, it sets the tuple $T \leftarrow (\text{epk}_{i_*}^{s_*}, \text{pk}_{j_*})$. If, afterward, $\mathcal{B}$ needs to compute the hash of an element containing both $(\overline{\text{epk}_{i_*}^{s_*}})^{\text{esk}_{i_*}^{s_*}}$ and $\text{pk}_{j_*}^{\text{esk}_{i_*}^{s_*}}$ using the random oracle, it replaces $(\overline{\text{epk}_{i_*}^{s_*}})^{\text{esk}_{i_*}^{s_*}}$ by $\perp$ and $\text{pk}_{j_*}^{\text{esk}_{i_*}^{s_*}}$ by T , and labels the query as ' $\perp$ -incomplete' when storing it in the list of random oracle queries (but still returns a random value corresponding to the query). For the sake of consistency with the random oracle, if at any time $\mathcal{B}$ generates an ephemeral key epk_i^s such that $\text{epk}_i^s = \overline{\text{epk}_{i_*}^{s_*}}$, then $\perp$ is replaced by $(\text{epk}_{i_*}^{s_*})^{\text{esk}_i^s}$ in the $\perp$ -incomplete query, and the label is replaced by 'incomplete'.

- Else if $\text{role}_{i_*}^{s_*} = \text{responder}$, $\mathcal{B}$ proceeds as in the previous case, except that it does not compute the key K and does not decrypt the ciphertext C it receives. Instead, it assumes that the decryption of C is pk_{j_*} . Recall that according to game G_1 , the adversary must send to $\pi_{i_*}^{s_*}$ a ciphertext correctly formed of the key pk_{j_*} in order to keep a non-negligible advantage of winning. The game is therefore correctly simulated to $\mathcal{A}$ as long as the adversary keeps its advantage.

We note that in the two cases, $T = (A, B)$, and that there is at most one unique $\perp$ -incomplete query.

- Each time $\mathcal{B}$ must process a query to the random oracle that matches the pattern of a previous query labelled '*incomplete*' (i.e. all elements that are not tuples match in the queries), it checks whether each group element Z corresponding to a tuple $T = (\text{epk}, \text{pk}_{j_*})$ satisfies $\text{stDH}_b(\text{epk}, Z) = 1$. If so, it returns the value associated with the incomplete query; otherwise, it processes the query as usual. For instance, if $\mathcal{B}$ simulates the first response of P_{j_*} to the first message send by P_i in the session π_i^s , when generating K , it will replace the query $(X^v \| X^y \| X \| Y)$ with the incomplete query $((X, V) \| X^y \| X \| Y)$ to the random oracle H . If, subsequently, $\mathcal{A}$ sends a query $(Z \| X^y \| X \| Y)$, $\mathcal{B}$ will check whether $\text{stDH}_v(X, Z) = 1$, and if so, it will return K (otherwise it will treat $(Z \| X^y \| X \| Y)$ as a standard query to a random oracle).
- Each time $\mathcal{B}$ must process a query to the random oracle that matches the pattern of the query labeled ' $\perp$ -incomplete' (i.e. all elements that are not tuples or $\perp$ match in the queries), it checks whether the group element Z of the query corresponding to the tuple $T = (A, B)$ in the incomplete query satisfies $\text{stDH}_b(A, Z) = 1$. If so, it aborts the experiment and returns Z ; otherwise, it processes the query as usual. As a result, it is never necessary to know the value replaced by $\perp$.
- To maintain consistency in responses, when $\mathcal{B}$ stores a new incomplete query, it must also check that no previous query matches this incomplete query in the same way; if so, it must return the corresponding hash of that previous query.
- In order to take into account all cases where the adversary sends a query containing $g^{\text{esk}_{i_*}^{s_*} \text{sk}_{j_*}}$ (possibly by chance) to the random oracle, $\mathcal{B}$ tests, for each element Z contained in each query, whether $\text{stDH}_b(A, Z) = 1$.
- If, at any other point, $\mathcal{B}$ tests $\text{stDH}_b(A, Z) = 1$, it aborts the experiment and returns Z .
- If $\mathcal{B}$ must process the query j_* to RevLTK , $\mathcal{B}$ aborts the game ($\mathcal{A}$ can no longer win G_1 because P_{j_*} must not be corrupted for event E to occur on j_*).

In this way, G_1 is perfectly simulated for $\mathcal{A}$ except for abortions when $\mathcal{A}$ can no longer win the game or when $\mathcal{B}$ is guaranteed to win their SDH experiment, and therefore:

$$\Pr[G_1 \text{ returns } 1] \leq \text{Adv}^{\text{SDH}}(\lambda).$$

Finally, the following concludes the proof:

$$\Pr \left[\text{Exp}_{H^{\text{SDH}}, \mathcal{A}}^{\text{MITM-privacy}}(\lambda) = 1 \wedge E \right] \leq \ell \mu^2 \text{Adv}^{\text{SDH}}(\lambda).$$

Proof (Theorem 1). We use the following sequence of games.

Game G_0 : This is the experiment $\text{Exp}_{\Pi_{\text{SDH}}, \mathcal{A}}^{\text{MITM-privacy}}(\lambda)$. We have that:

$$\Pr \left[\text{Exp}_{\Pi_{\text{SDH}}, \mathcal{A}}^{\text{MITM-privacy}}(\lambda) = 1 \right] = \Pr [G_0 \text{ returns } 1].$$

Game G_1 : Same as G_0 except that at the end of the game, if $\mathcal{A}$ wins according to the conditions of G_0 , and if for some tuple (s, i_*, j_*, R) , P_{i_*} and P_{j_*} are never corrupted, $\text{Pid}_{i_*}^s = j_*$, and the R^{th} query to the RO contains the value $g^{\text{esk}_{i_*}^s \text{sk}_{j_*}}$ (where $\text{esk}_{i_*}^s$ is the ephemeral key used in $\pi_{i_*}^s$), then G_1 sets $\text{abort}_1 = \text{true}$, aborts the game, and returns a random bit. According to Lemma 1, we have:

$$|\Pr [G_0 \text{ returns } 1] - \Pr [G_1 \text{ returns } 1]| \leq \Pr [\text{abort}_1] \leq \ell \mu^2 \text{Adv}^{\text{SDH}}(\lambda).$$

Game G_2 : G_2 is the same as G_1 except that if in two distinct sessions, the ephemeral public keys randomly chosen by the oracles are the same, then G_2 sets $\text{abort}_2 = \text{true}$, aborts and returns a random bit. At most, there is $\ell \mu$ sessions during the game, we deduce that:

$$|\Pr [G_1 \text{ returns } 1] - \Pr [G_2 \text{ returns } 1]| \leq \Pr [\text{abort}_2] \leq (\ell \mu)^2 / |\mathbb{G}|.$$

Game G_3 : G_3 is the same as G_2 except that if the RO returns the same random element for two distinct queries, then G_3 sets $\text{abort}_3 = \text{true}$, aborts and returns a random bit. We set q as the number of queries to the random oracle during G_2 , we have:

$$|\Pr [G_2 \text{ returns } 1] - \Pr [G_3 \text{ returns } 1]| \leq \Pr [\text{abort}_3] \leq q^2 / |\mathcal{K}|.$$

From this step, the abort conditions in games 2 and 3 ensure that the key K used in session $\pi_{i|j}^1$ if $P_{i|j}$ is the initiator is used only once for encryption (it can be used for decryption in the other session that matches the first two interactions and therefore uses the same ephemeral keys; this session is unique if it exists according to G_2). Furthermore, according to the condition of G_1 , if the adversary submits the query to obtain K to the random oracle, then its winning probability drops to $1/2$. K is therefore perfectly random from their point of view as long as they still have a non-zero advantage in winning the game.

Game G_4 : G_4 is the same as G_3 except that if the role of $P_{i|j}$ in $\pi_{i|j}^1$ is initiator, then it computes $C \leftarrow \text{Enc}_K(\perp)$ instead of computing $C \leftarrow \text{Enc}_K(U)$ at the second interaction. If during the simulation of an oracle the challenger has to decrypt C with the key K , they do not decrypt C and act as if the decryption returned U . Furthermore, if during the simulation of an oracle the challenger has to decrypt a ciphertext $C' \neq C$ with the key K , they act as if the decryption had failed.

We claim that:

$$|\Pr [G_3 \text{ returns } 1] - \Pr [G_4 \text{ returns } 1]| \leq \text{Adv}_{\text{SE}}^{\text{AE}}(\lambda).$$

We prove this claim by reduction. Assume that there exists an adversary $\mathcal{A}$ such that $|\Pr [G_3 \text{ returns } 1] - \Pr [G_4 \text{ returns } 1]|$ is non-negligible, we build the following AE adversary $\mathcal{B}$. $\mathcal{B}$ simulates the game G_3 to $\mathcal{A}$, except that:

- In $\pi_{i|j}^1$, if $P_{i|j}$ is the initiator, when $\mathcal{B}$ must determine K with the random oracle for the query $(V^x \| Y^x \| X \| Y)$, it associates the query with the label 'undefined'. Then, instead of computing $C \leftarrow \text{Enc}_K(U)$, it sends $(M_0, M_1) = (U, \perp)$ (where U is the public key of $P_{i|j}$ to its challenger and receives the challenge ciphertext ctxt . It sets $C \leftarrow \text{ctxt}$.
- Each time that $\mathcal{B}$ should compute the hash of query labeled with 'undefined' to determine the value of a key K , it uses its encryption/decryption oracles $\text{Enc}_K(\cdot)$ and $\text{Dec}_K(\cdot)$ instead of directly using the hash K (If $\mathcal{B}$ must decrypt C , it cannot use the decryption oracle; instead, it considers that the decryption of C is U).
- If at any point the adversary $\mathcal{A}$ requests or has requested a query labeled with 'undefined' to the random oracle, $\mathcal{B}$ aborts the game and returns a random bit. Note that this behaviour is consistent with the condition introduced in game G_1 .
- $\mathcal{B}$ returns 1 if and only if $\mathcal{A}$ wins the game.

In this way, if $b = 0$, G_3 is perfectly simulated at $\mathcal{A}$, else G_4 is perfectly simulated at $\mathcal{A}$, except for abortions when $\mathcal{A}$ has no longer advantage, and therefore, the following concludes the proof of the claim:

$$\Pr[G_3 \text{ returns } 1] = \Pr[\text{Exp}_{\text{SE},0,\mathcal{B}}^{\text{AE}}(\lambda) = 1];$$

$$\Pr[G_4 \text{ returns } 1] = \Pr[\text{Exp}_{\text{SE},1,\mathcal{B}}^{\text{AE}}(\lambda) = 1].$$

Game G_5 : G_5 is the same as G_4 except that at the end of the game, if the oracle RevSessKeyPriv generated a random session key k^* equals to either another session key randomly generated by the same oracle or an output of the random oracle, then G_5 sets $\text{abort}_5 = \text{true}$, aborts and returns a random bit. We set q (resp. q_r) as the number of queries to the random oracle (resp. RevSessKeyPriv) during G_4 , we have:

$$|\Pr[G_4 \text{ returns } 1] - \Pr[G_5 \text{ returns } 1]| \leq \Pr[\text{abort}_5] \leq q_r(q + q_r)/|\mathcal{K}|.$$

At this stage, we observe that:

- If $\pi_{i|j}^1$ is the initiator, the keys and the ciphertext produced by $\pi_{i|j}^1$ no longer contain any information about the real identity of $P_{i|j}$ according (the keys generated by the RO appear perfectly random to the adversary according to G_1 , and the ciphertext contains $\perp$ according to G_4). If it is the responder, no exchanged message depends on the identity of $P_{i|j}$ anymore (according to G_1). It is therefore no longer possible to directly exploit messages exchanged by $\pi_{i|j}^1$ to deduce its identity. Furthermore, if another oracle (for an uncorrupted party) is initialized with $\text{Pid} = i|j$ (i.e., with the initiator role), G_1 guarantees that the encryption and session keys used by this oracle appear to have been generated randomly from the adversary's point of view.

- If $\pi_{i|j}^1$ has a matching conversation, then it is with a unique partner oracle (because the ephemeral keys are all different according to G_2). In this case, according to the definition of the `RevSessKeyPriv` oracle, the revealing of the session key of the $\pi_{i|j}^1$ and the oracle that has a matching conversation with it will be the same (even if one of the parties reject the session). It will therefore not be possible to use `RevSessKeyPriv` as a distinguisher to learn information about $\pi_{i|j}^1$ identity. This prevents trivial attacks where the adversary engages an initiator oracle with partner P_i , and forwards messages from that oracle to $\pi_{i|j}^1$ (in this case, $\pi_{i|j}^1$ would accept the session if its identity is P_i , and fail if its identity is P_j).
- In all other cases, regardless of $P_{i|j}$ actual identity, if no oracle π_k^s has a matching conversation with $\pi_{i|j}^1$, then no oracle has the same context as $\pi_{i|j}^1$ and therefore no oracle shares the same session key, because the context is hashed to compute the session key, and all hash values are different according to G_3 . It should also be noted that, by definition, even if a session oracle refuses the session after receiving the last message C , `RevSessKeyPriv` returns a random key that is different from those of the other oracles according to G_5 . Thus, the key returned by `RevSessKeyPriv` on $\pi_{i|j}^1$ will be different from the key returned on any other session oracle. It is therefore not possible to use `RevSessKeyPriv` to distinguish the identity of $P_{i|j}$ in this case. This prevents attacks where the adversary would try to forge a end-of-session cipher C' containing the key of P_i in order to replace the real cipher C containing the key of $P_{i|j}$, in order to test whether the partner oracle of $\pi_{i|j}^1$ returns the same session key as $\pi_{i|j}^1$ (which would mean that $P_{i|j} = P_i$) or not.

Recall the winning conditions for the MITM-privacy experiment: P_i and P_j are never corrupted, and $\text{Pid}_{i|j}^1 = \perp$ or $\text{Pid}_{i|j}^1 = k$ while P_k is never corrupted. If $\text{Pid}_{i|j}^1 = \perp$, then the session key revealed by `RevSessKeyPriv` is random by definition. If $\text{Pid}_{i|j}^1 = k$ while P_k is never corrupted, then the session key of $\pi_{i|j}^1$ appears perfectly random to $\mathcal{A}$ according to G_1 . As long as game G_5 has not been aborted, $\mathcal{A}$ has no further information that can be exploited to guess the identity of $P_{i|j}$, neither by exploiting messages sent/received by $\pi_{i|j}^1$, nor by comparing session keys, so its advantage is zero, and it cannot guess better than guessing at random:

$$\Pr[G_5 \text{ returns } 1] = 1/2.$$

By considering that $|\mathbb{G}| > 2^\lambda$ and $|\mathcal{K}| > 2^\lambda$, we can therefore conclude that:

$$\left| \Pr \left[\text{Exp}_{\Pi_{\text{SDH}}, \mathcal{A}}^{\text{MITM-privacy}}(\lambda) = 1 \right] - 1/2 \right| \leq \ell \mu^2 \text{Adv}^{\text{SDH}}(\lambda) + ((\mu \ell)^2 + q^2 + q_r(q + q_r))/2^\lambda + \text{Adv}_{\text{SE}}^{\text{AE}}(\lambda).$$

Theorem 2. *Our protocol Π_{SDH} instantiated with a secure AE scheme is forward-private under the SDH assumption in the ROM.*

The proof of forward-privacy follows the same idea. Without interfering in the communications between two uncorrupted users, the adversary cannot deduce

g^{xy} , and the key K therefore appears to be completely random, perfectly hiding the identity U .

Proof (Theorem 2). We use the following sequence of games.

Game G_0 : This is the experiment $\text{Exp}_{\Pi^{\text{SDH}}, \mathcal{A}}^{\text{Forward-privacy}}(\lambda)$. We have that:

$$\Pr \left[\text{Exp}_{\Pi^{\text{SDH}}, \mathcal{A}}^{\text{Forward-privacy}}(\lambda) = 1 \right] = \Pr [G_0 \text{ returns } 1].$$

Game G_1 : G_1 is the same as G_0 except that the challenger picks $(i_*, j_*, k_*, r_*) \xleftarrow{\$} [\mu]^3 \times [\ell]$, and if $i|j \neq i_*|j_*$ or if $\pi_{k_*}^{r_*}$ is not the partner oracle of $\pi_{i_*|j_*}^1$, then G_1 aborts and returns a random bit (note that according to the winning condition of the experiment, the returned oracle $\pi_{i|j}^1$ must have a partner oracle at the end of the game). We have that:

$$\Pr [G_0 \text{ returns } 1] \leq \ell \mu^3 \Pr [G_1 \text{ returns } 1].$$

Game G_2 : This hop is identical to that from G_2 to G_3 in the previous proof.

Game G_3 : G_3 is the same as G_2 except that if the adversary sends a query containing $g^{\text{esk}_{i_*|j_*}^1 \text{esk}_{k_*}^{r_*}}$ (where $\text{esk}_{i_*|j_*}^1$ and $\text{esk}_{k_*}^{r_*}$ are the ephemeral keys x and y used in the two sessions) to the random oracle, then G_3 sets $\text{abort}_3 = \text{true}$, aborts and returns a random bit. We claim that:

$$|\Pr [G_2 \text{ returns } 1] - \Pr [G_3 \text{ returns } 1]| \leq \Pr [\text{abort}_3] \leq \text{Adv}^{\text{SDH}}(\lambda)$$

We prove the claim by reduction. Let $\mathcal{A} \in \text{Poly}(\lambda)$ be an adversary such that $\Pr [\text{abort}_3]$ is non-negligible. We build the following algorithm $\mathcal{B} \in \text{Poly}(\lambda)$ against SDH. $\mathcal{B}(A, B)$ simulates G_2 to $\mathcal{A}$, except that:

- $\mathcal{B}$ sets $\text{epk}_{i_*|j_*}^1 \leftarrow A$ and $\text{epk}_{k_*}^{r_*} \leftarrow B$, where $\text{epk}_{i_*|j_*}^1$ and $\text{epk}_{k_*}^{r_*}$ are the ephemeral public keys used in $\pi_{i_*|j_*}^1$ and $\pi_{k_*}^{r_*}$, noted in the protocol X (resp. Y) for the initiator (resp. responder).
- Recall that $\mathcal{A}$ must ensure that $\pi_{i_*|j_*}^1$ and $\pi_{k_*}^{r_*}$ are partners (and so $\text{Pid}_{i_*|j_*}^1 = k_*$) in order to maintain a non-negligible advantage according to G_1 . As soon as this condition can no longer be satisfied, $\mathcal{B}$ aborts the game by returning a random bit. When it simulates $\pi_{i_*|j_*}^1$, $\mathcal{B}$ computes $(\text{epk}_{i_*|j_*}^1)^{\text{sk}_{k_*}}$ instead of $\text{pk}_{k_*}^{\text{esk}_{i_*|j_*}^1}$. Moreover, when $\mathcal{B}$ should compute $(\text{epk}_{k_*}^{r_*})^{\text{esk}_{i_*|j_*}^1}$, it replaces it with $T = (A, B)$. It then labels the queries to the random oracle containing T as 'incomplete' when storing it in the list of random oracle queries (but still returns a random value corresponding to the query).
- In a similar way, when it simulates $\pi_{k_*}^{r_*}$, $\mathcal{B}$ computes $(\text{epk}_{k_*}^{r_*})^{\text{sk}_{i_*|j_*}}$ instead of $\text{pk}_{i_*|j_*}^{\text{esk}_{k_*}^{r_*}}$. Moreover, when $\mathcal{B}$ should compute $(\text{epk}_{i_*|j_*}^1)^{\text{esk}_{k_*}^{r_*}}$, it replaces it with $T = (A, B)$. It then labels the queries to the random oracle containing T as 'incomplete' when storing it in the list of random oracle queries (but still returns a random value corresponding to the query).

- Each time $\mathcal{B}$ must process a query to the random oracle that matches the pattern of a previous query labelled '*incomplete*', it checks whether the group element Z of the query corresponding to the tuple $T = (A, B)$ in the incomplete query satisfies $\text{stDH}_b(A, Z) = 1$. If so, it aborts the experiment and returns Z , otherwise, it processes the query as usual.
- To maintain consistency in responses, when $\mathcal{B}$ stores a new incomplete query, it must also check that no previous query matches this incomplete query; if so, it must return the corresponding hash of that previous query.

To keep an advantage in the game, the adversary must not interfere with communication between $\pi_{i_*|j_*}^1$ and $\pi_{k_*}^{r*}$ (they must be exclusive partner oracles), and therefore cannot modify the ephemeral keys exchanged. In this way, G_2 is perfectly simulated at $\mathcal{A}$ except for abortions when $\mathcal{A}$ can no longer win the game or when $\mathcal{B}$ is guaranteed to win their SDH experiment, and therefore $\Pr[\text{abort}_3] \leq \text{Adv}^{\text{SDH}}(\lambda)$, which concludes the proof of the claim.

From this step, the abort conditions in games 2 and 3 ensure that while the adversary keeps an advantage, the keys generated by the session seem to their fully random, and are never reused in another session (except the key shared with $\pi_{k_*}^{r*}$). As a result, the latest information that the adversary can exploit about the identity of $P_{i|j}$ is the message encrypted with the key K , which contains its public key (if $P_{i|j}$ plays the role of initiator).

Game G_4 and G_5 : These hops are identical to those from G_3 to G_4 and from G_4 to G_5 in the previous proof.

At this stage, we observe that:

- If $\pi_{i_*|j_*}^1$ is the initiator, the keys and the ciphertext produced by $\pi_{i_*|j_*}^1$ no longer contain any information about the real identity of $P_{i_*|j_*}$ (the keys generated by the RO appear perfectly random to the adversary according to G_3 , and the ciphertext contains $\perp$ according to G_4). If it is the responder, no exchanged message with its partner oracle depends on the identity of $P_{i_*|j_*}$ anymore. It is therefore no longer possible to directly exploit messages exchanged by $\pi_{i_*|j_*}^1$ to deduce its identity. Furthermore, the winning conditions of the forward-privacy ensure that no oracle besides $\pi_{k_*}^{r*}$ may be instructed to start a protocol run with intended partner $P_{i_*|j_*}$.
- As in the previous proof, if $\pi_{i_*|j_*}^1$ has a matching conversation, then the revealing of the session key of $\pi_{i_*|j_*}^1$ and the oracle that has a matching conversation with it will be the same. In all other cases, regardless of $P_{i_*|j_*}$ actual identity, if no oracle π_k^s has a matching conversation with $\pi_{i_*|j_*}^1$, then no oracle shares the same session key. It is therefore not possible to use RevSessKeyPriv to distinguish the identity of $P_{i_*|j_*}$.

Recall the winning conditions for the forward-privacy experiment: the returned oracle $\pi_{i|j}^1$ has a partner oracle π_k^r at the end of the game, and no oracle besides

π_k^r may be instructed to start a protocol run with intended partner $P_{i|j}$. As long as game G_5 has not been aborted, $\mathcal{A}$ has no further information that can be exploited to guess the identity of $P_{i|j}$, neither by exploiting messages exchanged by $\pi_{i|j}^1$ and π_k^r , nor by comparing session keys, so its advantage is zero, and it cannot guess better than guessing at random:

$$\Pr[G_5 \text{ returns } 1] = 1/2.$$

By considering that $|\mathbb{G}| > 2^\lambda$ and $|\mathcal{K}| > 2^\lambda$, we can therefore conclude that:

$$\left| \Pr \left[\text{Exp}_{\Pi_{\text{SDH}}, \mathcal{A}}^{\text{Forward-privacy}}(\lambda) = 1 \right] - 1/2 \right| \leq \ell \mu^3 \left(\text{Adv}^{\text{SDH}}(\lambda) + (q + q^2 + q_r(q + q_r))/2^\lambda + \text{Adv}_{\text{SE}}^{\text{AE}}(\lambda) \right).$$

Theorem 3. *Our protocol Π_{SDH} instantiated with a secure AE scheme is key secret under the SDH assumption in the ROM.*

Recall that a very similar protocol (Π) has already been proven key secret in [6], the only difference being the use of the key $K = H(g^{xv} \| g^{xy} \| X \| Y)$ to encrypt the public key U (U is not a sensitive value for key-secrecy). Since the key K comes from the RO, it does not provide the adversary with any information about the values g^{xv} and g^{xy} , and is therefore of no use in guessing the session key. Since it essentially follows the same spirit as the proof of the protocol Π in [6], we have deferred it to the Appendix B.

5 Protocol Π_{CDH}

We are now building a variant of our protocol that can be proven under CDH, which is weaker than SDH and is generally considered more standard. This protocol uses signatures of knowledge of a discrete logarithm given a group element and a base. We first recall the formal definition of signature of knowledge.

Definition 7 (Signature Of Knowledge (SOK) [5]). *Let $\mathcal{R}$ be a binary relation and let $\mathcal{L}$ be a language such that $s \in \mathcal{L} \Leftrightarrow (\exists w, (s, w) \in \mathcal{R})$. A signature of knowledge for $\mathcal{L}$ is a couple of algorithms (SOK, Verify) such that the algorithm $\text{SOK}_m\{w : (s, w) \in \mathcal{R}\}$ outputs a signature sok of a message m using w , and the algorithm $\text{Verify}(s, m, \text{sok})$ outputs a bit b such that $\text{Verify}(s, m, \text{SOK}_m\{w : (s, w) \in \mathcal{R}\}) = 1$ for any w such that $(s, w) \in \mathcal{R}$. A SOK must guarantee the following properties:*

- **Extractability.** *There exists a polynomial time knowledge extractor Ext and a negligible function $\text{Adv}_{\text{SOK}}^{\text{ext}}(\lambda)$ such that for any algorithm $\mathcal{A}^{\text{Sim}(\cdot)}(\lambda)$ having access to a simulator that forges signatures for chosen statement and that outputs a fresh tuple (s, m, sok) with $\text{Verify}(s, m, \text{sok}) = 1$, the extractor $\text{Ext}^{\mathcal{A}}(\lambda)$ having access to $\mathcal{A}$ outputs w such that $(s, w) \in \mathcal{R}$ with probability at least $1 - \text{Adv}_{\text{SOK}}^{\text{ext}}(\lambda)$.*

- **(Perfect) Zero-knowledge.** A signature sok leaks no information, i.e., there exists a polynomial time algorithm Sim (called the simulator) such that $\text{SOK}_m\{w : (s, w) \in \mathcal{R}\}$ and $\text{Sim}(m, s)$ follow the same probability distribution for any (m, s) and any w such that $(s, w) \in \mathcal{R}$.

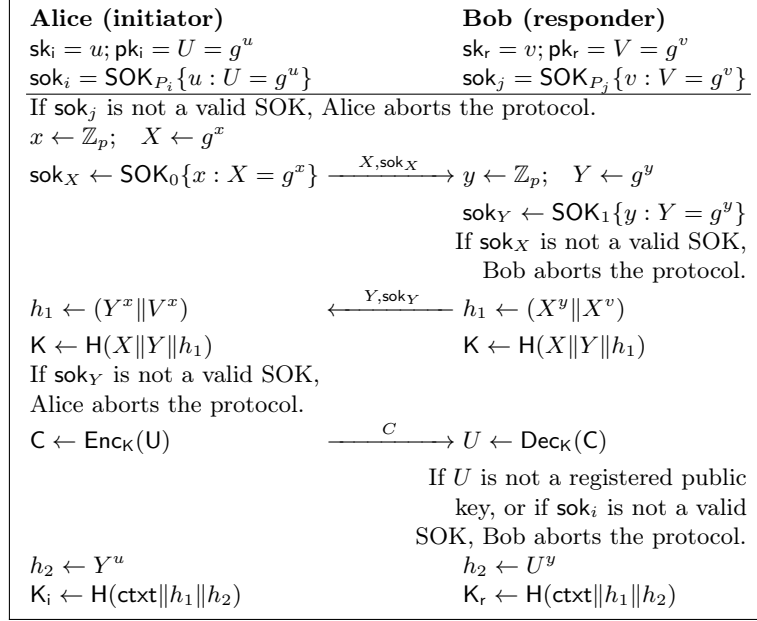


Fig. 3. Private authenticated key exchange protocol based on CDH (Π_{CDH}).

The SOK of a discrete logarithm can be instantiated by the Schnorr protocol [17] rendered interactive with the Fiat-Shamir heuristic [8], and where the message is hashed with the rest of the elements. In this case, $\text{SOK}_m\{w : W = g^w\}$ consists of choosing an r and returning $(R = g^r, \alpha = r + xH(g, W, R, m))$ (1 exponentiation), and $\text{Verify}((g, W), m, \text{sok})$ consists of verifying whether $g^\alpha = Ry^{H(g, W, R, m)}$ (2 exponentiations). Note that we implicitly consider in the SOK definition that any hash function will be simulated by a random oracle (RO). In particular, the Schnorr simulator Sim can program the random oracles (Sim picks (α, h) , computes $R = y^h/g^\alpha$, and then programs the random oracle such that $h = H(g, W, R, m)$). However, the algorithm $\mathcal{A}$ in extractability only has black-box access to the random oracles, so it cannot program the random oracles and therefore cannot simulate valid proofs on its own on arbitrary statements to extract the witness.

Our Π_{CDH} protocol, shown in Figure 3, is the same as the Π_{SDH} protocol, except that each user uses their long-term key to sign their identity, and uses

their ephemeral key to sign their role (with `initiator` = 0 and `responder` = 1) at each session. Each user must therefore generate a SOK and verify a SOK at each session, which adds 3 exponentiations per user. Note that it is also necessary to verify the SOK of the long-term key, but this can be performed only once (when verifying the certificate of this key) and does not need to be repeated for other sessions with the same partner. The security of this protocol is given in the following theorem.

Theorem 4. *Our protocol Π_{CDH} instantiated with a secure AE scheme and the Schnorr SOK is MITM-private, forward-private and key-secret under the CDH assumption in the ROM.*

The detailed proof of this theorem is given in Appendix C. The proofs of the three properties follow essentially the same structure as those of the previous protocol, except for the CDH reductions. In the previous proofs, during these reductions, the SDH challenge (A, B) was used to instantiate a long-term key and an ephemeral key or two ephemeral keys. Since, before being hashed, these keys are used to compute Diffie-Hellman keys with several different key shares, and since it is sometimes impossible to compute these keys without knowing the discrete logarithm of A or B , we stored the key shares that would have had to be merged in the random oracle queries, then used the `stDH` oracle to check whether the attacker was sending the corresponding queries. For example, if an adversary's query corresponds to an incomplete query requiring the Diffie-Hellman key of key shares (X, B) , we checked whether `stDHb`(X, Z), where Z is the element sent by the adversary. In Π_{CDH} , in similar reductions, while (X, sok_X) is a fresh pair of ephemeral key and SOK (not already outputted by the challenger), we can instead use the knowledge extractor on `sokX` to find x and compute $Z = B^x$. However, it should be noted that, since the discrete logarithm of A and B is unknown, the corresponding signatures must be simulated (these signatures are therefore non-extractable). Thus, the adversary could reuse these elements and signatures without knowing the corresponding discrete logarithms. However, since the signature signs the user's role (for ephemeral key) or the user's identity (for long-term key), they can only be reused in the same context. For example, if an oracle sends the ephemeral key A with a signature of the role `initiator`, the adversary cannot send the same ephemeral key back to the oracle, since they are supposed to play the role of the responder. For similar reasons, the adversary cannot register a user using such an ephemeral key as a public key. This prevents cases that cannot be simulated in the reductions.

6 Conclusion and Future Work

In this article, we have given two efficient round-optimal private key exchange protocols and proved their security in the ROM under SDH and CDH. There are many directions in which future research could go. Firstly, our privacy proofs are not tight, and it would be interesting to improve them and prove the optimal loss reduction that can be achieved for privacy. Secondly, it would be interesting to

investigate whether a similar approach could be taken with quantum-resistant primitives, such as in key exchange based on key encapsulation mechanisms. Finally, since our protocols are very similar to Noise, which is used in real-world applications, it would be interesting to see what using our protocols would entail in practice (e.g. for Signal [11], WhatsApp [18] or Wireguard[7]).

Acknowledgments

This work has been supported by the French National Research Agency through the PRIVASIQ project (ANR-23-CE39-0008).

References

1. Abdalla, M., Bellare, M., Rogaway, P.: The oracle diffie-hellman assumptions and an analysis of DHIES. In: Naccache, D. (ed.) Topics in Cryptology - CT-RSA 2001, The Cryptographer's Track at RSA Conference 2001, San Francisco, CA, USA, April 8-12, 2001, Proceedings. Lecture Notes in Computer Science, vol. 2020, pp. 143–158. Springer (2001). https://doi.org/10.1007/3-540-45353-9_12, https://doi.org/10.1007/3-540-45353-9_12
2. Aiello, W., Bellovin, S.M., Blaze, M., Canetti, R., Ioannidis, J., Keromytis, A.D., Reingold, O.: Just fast keying: Key agreement in a hostile internet. ACM Trans. Inf. Syst. Secur. **7**(2), 242–273 (2004). <https://doi.org/10.1145/996943.996946>, <https://doi.org/10.1145/996943.996946>
3. Arfaoui, G., Bultel, X., Fouque, P., Nedelcu, A., Onete, C.: The privacy of the TLS 1.3 protocol. IACR Cryptol. ePrint Arch. p. 749 (2019), <https://eprint.iacr.org/2019/749>
4. Canetti, R., Krawczyk, H.: Security analysis of ike's signature-based key-exchange protocol. In: Yung, M. (ed.) Advances in Cryptology - CRYPTO 2002, 22nd Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 2002, Proceedings. Lecture Notes in Computer Science, vol. 2442, pp. 143–161. Springer (2002). https://doi.org/10.1007/3-540-45708-9_10, https://doi.org/10.1007/3-540-45708-9_10
5. Chase, M., Lysyanskaya, A.: On signatures of knowledge. In: Dwork, C. (ed.) Advances in Cryptology - CRYPTO 2006, 26th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 2006, Proceedings. Lecture Notes in Computer Science, vol. 4117, pp. 78–96. Springer (2006). https://doi.org/10.1007/11818175_5, https://doi.org/10.1007/11818175_5
6. Cohn-Gordon, K., Cremers, C., Gjosteen, K., Jacobsen, H., Jager, T.: Highly efficient key exchange protocols with optimal tightness - enabling real-world deployments with theoretically sound parameters. IACR Cryptol. ePrint Arch. p. 737 (2019), <https://eprint.iacr.org/2019/737>
7. Dowling, B., Paterson, K.G.: A cryptographic analysis of the wireguard protocol. IACR Cryptol. ePrint Arch. p. 80 (2018), <http://eprint.iacr.org/2018/080>
8. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Odlyzko, A.M. (ed.) Advances in Cryptology - CRYPTO '86, Santa Barbara, California, USA, 1986, Proceedings. Lecture Notes in Computer Science, vol. 263, pp. 186–194. Springer (1986). https://doi.org/10.1007/3-540-47721-7_12, https://doi.org/10.1007/3-540-47721-7_12

9. Gamal, T.E.: A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Trans. Inf. Theory* **31**(4), 469–472 (1985). <https://doi.org/10.1109/TIT.1985.1057074>, <https://doi.org/10.1109/TIT.1985.1057074>
10. Lyu, Y., Liu, S., Han, S., Gu, D.: Privacy-preserving authenticated key exchange in the standard model. In: *ASIACRYPT 2022*. pp. 210–240. Springer (2022)
11. Marlinspike, M., Perrin, T.: The x3dh key agreement protocol. <https://signal.org/docs/specifications/x3dh/> (2016), signal Foundation Specification
12. Paterson, K.G., Ristenpart, T., Shrimpton, T.: Tag size does matter: Attacks and proofs for the TLS record protocol. In: Lee, D.H., Wang, X. (eds.) *Advances in Cryptology - ASIACRYPT 2011 - 17th International Conference on the Theory and Application of Cryptology and Information Security*, Seoul, South Korea, December 4–8, 2011. Proceedings. Lecture Notes in Computer Science, vol. 7073, pp. 372–389. Springer (2011). https://doi.org/10.1007/978-3-642-25385-0_20, https://doi.org/10.1007/978-3-642-25385-0_20
13. Ramacher, S., Slamanig, D., Weninger, A.: Privacy-preserving authenticated key exchange: Stronger privacy and generic constructions. In: Bertino, E., Schulmann, H., Waidner, M. (eds.) *Computer Security - ESORICS 2021 - 26th European Symposium on Research in Computer Security*, Darmstadt, Germany, October 4–8, 2021, Proceedings, Part II. Lecture Notes in Computer Science, vol. 12973, pp. 676–696. Springer (2021). https://doi.org/10.1007/978-3-030-88428-4_33, https://doi.org/10.1007/978-3-030-88428-4_33
14. Ramacher, S., Slamanig, D., Weninger, A.: Privacy-preserving authenticated key exchange: Stronger privacy and generic constructions. *IACR Cryptol. ePrint Arch.* p. 1338 (2022), <https://eprint.iacr.org/2022/1338>
15. Schäge, S., Schwenk, J., Lauer, S.: Privacy-preserving authenticated key exchange and the case of ikev2. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) *Public-Key Cryptography - PKC 2020 - 23rd IACR International Conference on Practice and Theory of Public-Key Cryptography*, Edinburgh, UK, May 4–7, 2020, Proceedings, Part II. Lecture Notes in Computer Science, vol. 12111, pp. 567–596. Springer (2020). https://doi.org/10.1007/978-3-030-45388-6_20, https://doi.org/10.1007/978-3-030-45388-6_20
16. Schnorr, C.: Efficient identification and signatures for smart cards. In: Brassard, G. (ed.) *Advances in Cryptology - CRYPTO '89, 9th Annual International Cryptology Conference*, Santa Barbara, California, USA, August 20–24, 1989, Proceedings. Lecture Notes in Computer Science, vol. 435, pp. 239–252. Springer (1989). https://doi.org/10.1007/0-387-34805-0_22, https://doi.org/10.1007/0-387-34805-0_22
17. Schnorr, C.: Efficient signature generation by smart cards. *J. Cryptol.* **4**(3), 161–174 (1991). <https://doi.org/10.1007/BF00196725>, <https://doi.org/10.1007/BF00196725>
18. WhatsApp Inc., Open Whisper Systems: Whatsapp encryption overview. <https://www.whatsapp.com/security/WhatsApp-Security-Whitepaper.pdf> (2016), technical Whitepaper
19. Zhao, Y.: Identity-concealed authenticated encryption and key exchange. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, Vienna, Austria, October 24–28, 2016.

pp. 1464–1479. ACM (2016). <https://doi.org/10.1145/2976749.2978350>,
<https://doi.org/10.1145/2976749.2978350>

A Key-Security Experiment

Definition 8 (Freshness). An oracle π_i^s is fresh if (i) $\Psi_i^s = \text{Accept}$, (ii) $\text{RevSessKey}(i, s)$ has not been issued; (iii) no query $\text{RevSessKey}(j, t)$ has been issued, where π_j^t is a partner of π_i^s ; and (iv) $P_{\text{Pid}_i^s}$ was not corrupted before π_i^s accepted if π_i^s has an origin-oracle, and not corrupted at all if π_i^s has no origin-oracle.

Key secrecy. The oracle $\text{Test}(i, s)$ is defined as follows: if $\Psi_i^s \neq \text{Accept}$, return $\perp$. Else, return k_b where $k_0 = k_i^s$, and $k_1 \xleftarrow{\$} \mathcal{K}$ is a random key. After this query, oracle π_i^s is said to be tested. The experiment $\text{Exp}_{\text{PPAKE}, \mathcal{A}}^{\text{key-secret}}(\lambda)$ is defined as follows:

1. Let μ be the number of parties in the game and ℓ the number of sessions per user. $\mathcal{C}$ begins by drawing a random bit $b \xleftarrow{\$} \{0, 1\}$ and generating key pairs $\{(\text{sk}_i, \text{pk}_i) \mid 1 \leq i \leq \mu\}$ as well as oracles $\{\pi_i^s \mid 1 \leq i \leq \mu, 1 \leq s \leq \ell\}$.
2. $\mathcal{C}$ now runs $\mathcal{A}$, providing all the public keys as input. During its execution, $\mathcal{A}$ may adaptively issue $\text{Send}(i, s, m)$, $\text{RevLTK}(i)$, $\text{RevSessKeySec}(i, s)$ and $\text{RegisterLTK}(i, \text{pk}_i)$ queries any number of times and the $\text{Test}(i, s)$ query once.
3. The game ends when $\mathcal{A}$ terminates with the output b' representing the guess of the secret bit b . If the tested oracle is not fresh, it outputs a random bit. Else, if the tested oracle remains fresh and $b' = b$, it outputs 1. Otherwise, it outputs 0.

We define the following three winning events in the game $\text{Exp}_{\text{PPAKE}, \mathcal{A}}^{\text{key-secret}}(\lambda)$:

1. **Event breakSound:** This event occurs if there exist two partner oracles π_i^s and π_j^t such that their session keys differ, i.e., $k_i^s \neq k_j^t$. In other words, two partner oracles compute different session keys.
2. **Event breakUnique:** This event occurs if there exists an oracle π_i^s that has two distinct partner oracles π_j^t and $\pi_{j'}^{t'}$. That is, one oracle has more than one partner oracle.
3. Let guessKE be the output of the game $\text{Exp}_{\text{PPAKE}, \mathcal{A}}^{\text{key-secret}}(\lambda)$. We define **breakKE** as the event where $\text{guessKE} = 1$.

Definition 9. Let PPAKE be a key exchange protocol. An adversary $\mathcal{A}$ breaks the key-secrecy of PPAKE if at least one of the events *breakSound*, *breakUnique*, or *breakKE* occurs during the game. The advantage of the adversary $\mathcal{A}$ in breaking the AKE security of protocol PPAKE is defined as:

$$\text{Adv}_{\text{PPAKE}, \mathcal{A}}^{\text{AKE}}(\lambda) = \max \left\{ \Pr[\text{breakSound}], \Pr[\text{breakUnique}], \left| \Pr[\text{breakKE}] - \frac{1}{2} \right| \right\}.$$

B Proof of key-secrecy for Π_{SDH}

Lemma 2. *Let E be the event: "at the end of the key-secrecy experiment, for some tuple (s, i, j, R) , P_j is never corrupted, $\text{Pid}_i^s = j$, and the R^{th} query to the RO contains the value $g^{\text{esk}_i^s \text{sk}_j}$, where esk_i^s is the ephemeral key used in π_i^s ". For any PPT adversary $\mathcal{A}$ playing the key-secrecy experiment on Π_{SDH} , the probability that E occurs and $\mathcal{A}$ wins the experiment is negligible under the SDH assumption in the ROM.*

Proof. This lemma can be proved in a manner similar to Lemma 1. The only notable difference is that in this lemma, we do not require that P_i be uncorrupted, which is never used in the proof of Lemma 1 (however, we do need P_j to be uncorrupted because in the reduction to SDH, the public key of P_j is replaced by one of the inputs of the SDH experiment and therefore the corresponding secret key is unknown).

Proof. We show that Π_{SDH} is key secret using the following sequences of games.

Game G_0 . In this game, the adversary interacts with honestly behaving oracles according to the key-secrecy security experiment. Oracles derive session keys exactly as specified in the protocol. The adversary's goal is to distinguish a real session key from a random one. We define:

$$\Pr[\text{break}_{\text{KE}}] = \Pr[G_0 \text{ returns } 1].$$

Game G_1 . This game is identical to G_0 except that the challenger aborts if at the end of the game, for some tuple (s, i, j, R) , P_j is not corrupted, $\text{Pid}_i^s = j$, and the R^{th} query to the RO contains the value $g^{\text{esk}_i^s \text{sk}_j}$, where esk_i^s is the ephemeral key used in π_i^s .

By Lemma 2, we have that:

$$|\Pr[G_0 \text{ returns } 1] - \Pr[G_1 \text{ returns } 1]| \leq \ell \mu^2 \text{Adv}^{\text{SDH}}(\lambda).$$

Game G_2 . This game is identical to G_1 except that the challenger aborts the experiment if two oracles in the same role (either initiators or responders) generate identical session contexts. This implies a collision in their ephemeral exponents, which is highly unlikely. Given at most ℓ sessions per user and μ users, this collision occurs with probability at most:

$$|\Pr[G_1 \text{ returns } 1] - \Pr[G_2 \text{ returns } 1]| \leq \frac{(\mu \ell)^2}{|\mathbb{G}|}.$$

Game G_3 . This game is identical to G_2 except that the challenger guesses a tuple (i_*, s_*, j_*, t_*) such that if the adversary $\mathcal{A}$ ever queries the **Test** oracle, it will be on $\pi_{i_*}^{s_*}$ and if that oracle has an origin oracle, it is $\pi_{j_*}^{t_*}$ (Note that t_* is unique, according to G_2). If the adversary $\mathcal{A}$ fails its guess, the game aborts. We have:

$$\Pr[G_2 \text{ returns } 1] \leq \mu^2 \ell^2 \cdot \Pr[G_3 \text{ returns } 1].$$

Game G_4 . This game is identical to G_3 except that the game sets $\text{abort}_4 = \text{true}$, aborts and returns a random bit if $\pi_{j_*}^{t_*}$ is the origin oracle of $\pi_{i_*}^{s_*}$, $\pi_{i_*}^{s_*}$ is fresh and the adversary has sent a query to the random oracle containing an element equal to $g^{\text{esk}_{i_*}^{s_*} \text{esk}_{j_*}^{t_*}}$ during the game. We claim that :

$$|\Pr[G_3 \text{ returns } 1] - \Pr[G_4 \text{ returns } 1]| \leq \Pr[\text{abort}_4] \leq \text{Adv}^{\text{SDH}}(\lambda).$$

We prove this claim by reduction. Let $\mathcal{A} \in \text{Poly}(\lambda)$ be an adversary such that $\Pr[\text{abort}_4]$ is non-negligible. We build the following algorithm $\mathcal{B} \in \text{Poly}(\lambda)$ against SDH. $\mathcal{B}(\mathcal{A}, \mathcal{B})$ simulates G_4 to $\mathcal{A}$, except that:

- $\mathcal{B}$ sets $\text{epk}_{j_*}^{t_*} \leftarrow B$, and sets $\text{epk}_{i_*}^{s_*} \leftarrow A$, where $\text{epk}_{i_*}^{s_*}$ and $\text{epk}_{j_*}^{t_*}$ are the ephemeral public keys, respectively, used in $\pi_{i_*}^{s_*}$ and $\pi_{j_*}^{t_*}$. If $\overline{\text{epk}_{i_*}^{s_*}} \neq \text{epk}_{j_*}^{t_*}$, then $\mathcal{B}$ aborts the game (in this case $\pi_{j_*}^{t_*}$ is not the origin oracle of $\pi_{i_*}^{s_*}$). Note that $\mathcal{B}$ does not know the respective discrete logarithms $\text{esk}_{j_*}^{t_*}$ and $\text{esk}_{i_*}^{s_*}$ of $\text{epk}_{j_*}^{t_*}$ and $\text{epk}_{i_*}^{s_*}$. It is therefore necessary to specify how $\mathcal{B}$ simulates the oracles that would use these values, i.e. oracle $\pi_{j_*}^{t_*}$ for $\text{esk}_{j_*}^{t_*}$ and the oracle $\pi_{i_*}^{s_*}$ for $\text{esk}_{i_*}^{s_*}$.
- In $\pi_{i_*}^{s_*}$, when $\mathcal{B}$ should compute $(\text{pk}_{\text{Pid}_{i_*}^{s_*}}^{s_*})^{\text{esk}_{i_*}^{s_*}}$, it computes $(\text{epk}_{i_*}^{s_*})^{\text{sk}_{\text{Pid}_{i_*}^{s_*}}^{s_*}}$ instead. In the case where $\mathcal{B}$ does not know the secret key $\text{sk}_{\text{Pid}_{i_*}^{s_*}}^{s_*}$ (i.e. $\text{pk}_{\text{Pid}_{i_*}^{s_*}}^{s_*}$ has been chosen by $\mathcal{A}$), then $\mathcal{B}$ aborts the game. Recall that if $\pi_{i_*}^{s_*}$ is fresh and has an origin oracle, then $\text{Pid}_{i_*}^{s_*}$ was not corrupted before $\pi_{i_*}^{s_*}$ accepts, so $\mathcal{B}$ knows the secret key of $\text{Pid}_{i_*}^{s_*}$.
- In $\pi_{i_*}^{s_*}$, when $\mathcal{B}$ should compute $(\text{epk}_{j_*}^{t_*})^{\text{esk}_{i_*}^{s_*}}$, it sets a tuple $T \leftarrow (A, B)$. If, afterward, $\mathcal{B}$ needs to compute the hash of an element containing $(\text{epk}_{j_*}^{t_*})^{\text{esk}_{i_*}^{s_*}}$ using the random oracle, it replaces it with T and labels the query as '*incomplete*' when storing it in the list of random oracle queries (but still returns a random value corresponding to the query). Each time $\mathcal{B}$ must process a query to the random oracle that matches the pattern of a previous query labelled '*incomplete*' containing T , it checks whether the group element Z of the query corresponding to the tuple $T = (A, B)$ in the incomplete query satisfies $\text{stDH}_b(A, Z) = 1$. If so, it aborts the experiment and returns Z , otherwise, it processes the query as usual.
- In $\pi_{j_*}^{t_*}$, when $\mathcal{B}$ should compute $(A')^{\text{esk}_{j_*}^{t_*}}$ for some A' , it sets a tuple $T' \leftarrow (A', B)$. If, afterward, $\mathcal{B}$ needs to compute the hash of an element containing $(A')^{\text{esk}_{j_*}^{t_*}}$ using the random oracle, it replaces it with T' and labels the query as '*incomplete*' when storing it in the list of random oracle queries (but still returns a random value corresponding to the query). Each time $\mathcal{B}$ must process a query to the random oracle that matches the pattern of a previous query labelled '*incomplete*' containing T' , it checks whether the group element Z' of the query corresponding to the tuple $T' = (A', B)$ in the incomplete query satisfies $\text{stDH}_b(A', Z') = 1$. If so, it replaces T' by Z' in the stored query (note that a query can contains several tuples T').

- To maintain consistency in responses, when $\mathcal{B}$ stores a new incomplete query, it must also check that no previous query matches this incomplete query in the same way; if so, it must return the corresponding hash of that previous query.

In this way, G_4 is perfectly simulated at $\mathcal{A}$ except for abortions when $\mathcal{A}$ can no longer win the game or when $\mathcal{B}$ is guaranteed to win their SDH experiment, and therefore $\Pr[\text{abort}_4] \leq \text{Adv}^{\text{SDH}}(\lambda)$,

Game G_5 . This game is identical to G_4 except that the challenger aborts the game if the RO outputs two times the same value on two different queries, or outputs k_1 . We set q as the number of queries to the RO, we have:

$$|\Pr[G_5 \text{ returns } 1] - \Pr[G_4 \text{ returns } 1]| \leq \Pr[\text{abort}_5] \leq (q+1)^2/|\mathcal{K}|.$$

Remember that in order to have a significant probability of winning, the tested oracle ($\pi_{i_*}^{s_*}$ according to G_3) must remain fresh, which means that the session keys of the tested oracle's partner oracles must not have been revealed. Note that given the G_2 game, since session keys are always different, the tested oracle can only have one partner, and given the G_5 game, two oracles that are not partners have a different context and necessarily have different session keys. As a result, $\mathcal{A}$ cannot deduce the tested session key by revealing the session keys of other oracles. Finally, for the adversary to maintain a significant advantage, they must be in one of these two situations required by freshness of the tested session $\pi_{i_*}^{s_*}$:

- $P_{\text{Pid}_{i_*}^{s_*}}$ was not corrupted before $\pi_{i_*}^{s_*}$ accepted if $\pi_{i_*}^{s_*}$ has an origin-oracle. In this case, the game G_4 ensures that the adversary must not have sent the key $(\overline{\text{epk}_{i_*}^{s_*}})^{\text{esk}_{i_*}^{s_*}}$ in a query to the RO, which is necessary to generate the oracle's session key.
- $P_{\text{Pid}_{i_*}^{s_*}}$ was not corrupted at all if $\pi_{i_*}^{s_*}$ has no origin-oracle. In this case, the game G_1 ensures that the adversary must not have sent the key $(\text{pk}_{\text{Pid}_{i_*}^{s_*}})^{\text{esk}_{i_*}^{s_*}}$ in a query to the RO, which is necessary to generate the oracle's session key.

In both cases, the session key of $\pi_{i_*}^{s_*}$ is therefore perfectly random from the point of view of $\mathcal{A}$. It therefore has no way of guessing whether the key returned by the test oracle is the real session key or a randomly chosen key. We deduce that:

$$\Pr[G_5 \text{ returns } 1] = 1/2.$$

Combining all bounds, we obtain:

$$\Pr[\text{break}_{\text{KE}}] \leq (\ell\mu)^2 \left(\frac{1}{|\mathbb{G}|} + 2 \cdot \text{Adv}^{\text{SDH}}(\lambda) + \frac{(q+1)^2}{|\mathcal{K}|} \right).$$

Furthermore, the probability of having two partner oracles at the same oracle is bounded by the probability that two oracles generate the same ephemeral key, and therefore:

$$\Pr[\text{break}_{\text{Unique}}] \leq \frac{(\mu\ell)^2}{|\mathbb{G}|}.$$

Finally, let us consider two partner oracles π_i^s and π_j^t , i.e., which share the same context and transcript, and satisfy $\text{Pid}_i^s = j$ and $\text{Pid}_j^t = i$. We will have:

$$\begin{aligned} k_i^s &= H(\text{ctxt} \| (\text{pk}_j)^{\text{esk}_i^s} \| (\text{epk}_j^t)^{\text{esk}_i^s} \| (\text{epk}_j^t)^{\text{sk}_i}) = H(\text{ctxt} \| g^{\text{sk}_j \text{esk}_i^s} \| g^{\text{esk}_j^t \text{esk}_i^s} \| g^{\text{esk}_j^t \text{sk}_i}); \\ k_j^t &= H(\text{ctxt} \| (\text{epk}_i^s)^{\text{sk}_j} \| (\text{epk}_i^s)^{\text{esk}_j^t} \| (\text{pk}_i)^{\text{esk}_j^t}) = H(\text{ctxt} \| g^{\text{sk}_j \text{esk}_i^s} \| g^{\text{esk}_j^t \text{esk}_i^s} \| g^{\text{esk}_j^t \text{sk}_i}) = k_i^s. \end{aligned}$$

We deduce that:

$$\Pr[\text{breakSound}] \leq \Pr[k_i^s \neq k_j^t] = 0.$$

To conclude, we obtain the following bound for the AKE advantage:

$$\text{Adv}_{\text{PPAKE}, \mathcal{A}}^{\text{AKE}}(\lambda) \leq (\ell\mu)^2 \left(\frac{1}{|\mathbb{G}|} + 2 \cdot \text{Adv}^{\text{SDH}}(\lambda) + \frac{(q+1)^2}{|\mathcal{K}|} \right).$$

C Proofs for Π_{CDH}

In the following proofs, sok_i denotes the signature of knowledge in the long-term public key of P_i . During a session simulated by the oracle π_i^s where P_i is the initiator, sok_i^s corresponds to sok_X and $\overline{\text{sok}}_i^s$ corresponds to sok_Y in Figure 3. When P_i is the responder, sok_i^s corresponds to sok_Y and $\overline{\text{sok}}_i^s$ corresponds to sok_X .

C.1 Proof of MITM-privacy for Π_{CDH}

Theorem 5. *Our protocol Π_{CDH} instantiated with a secure AE scheme and the Schnorr SOK is MITM-private under the CDH assumption in the ROM.*

As in the previous protocol, we prove MITM-privacy in two steps. First, we show that the adversary can never guess and send the Diffie-Hellman values of V^x and U^y to the RO instead of uncorrupted users (owners of V and U) under the CDH assumption in the random oracle. This result (Lemma 3) allows us to show that during the MITM-privacy experiment, the key K used by $P_{i||j}$ is perfectly random, and therefore that the identity encrypted in C can only be recovered by attacking the secure authenticated encryption scheme. Note that the second part of the proof is not impacted by the modifications in the protocol, and works as in the Π_{SDH} case.

Lemma 3. *Let E be the event: "at the end of the MITM-privacy experiment, for some tuple (s, i, j, R) , P_i and P_j are never corrupted, $\text{Pid}_i^s = j$, and the R^{th} query to the RO ends with concatenated group elements such that the last or second-to-last element is $g^{\text{esk}_i^s \text{sk}_j}$, where esk_i^s is the ephemeral key used in π_i^s ". For any PPT adversary $\mathcal{A}$ playing the MITM-privacy experiment on Π_{CDH} , the probability that E occurs and $\mathcal{A}$ wins the experiment is negligible under the CDH assumption in the ROM.*

Proof. We show that $\Pr \left[\text{Exp}_{\Pi_{\text{CDH}}, \mathcal{A}}^{\text{MITM-privacy}}(\lambda) = 1 \wedge E \right]$ is negligible using the following sequence of games.

Game G_0 : This is the experiment $\text{Exp}_{\Pi_{\text{CDH}}, \mathcal{A}}^{\text{MITM-privacy}}(\lambda)$ except that if E does not occur, the experiment returns 0. We have that:

$$\Pr \left[\text{Exp}_{\Pi_{\text{CDH}}, \mathcal{A}}^{\text{MITM-privacy}}(\lambda) = 1 \wedge E \right] = \Pr [G_0 \text{ returns } 1]$$

Game G_1 : G_1 is the same as G_0 except that the challenger replaces all the signatures of knowledge it has to generate by simulations. Since the Schnorr signature of knowledge is perfectly zero-knowledge, we have:

$$\Pr [G_0 \text{ returns } 1] = \Pr [G_1 \text{ returns } 1]$$

Game G_2 : G_2 is the same as G_1 except that the challenger runs the knowledge extractor on the fresh signatures of knowledge generated by the adversary and stores the corresponding pair of element/discrete logarithm when simulating the session oracles. If one extraction fails, it aborts the game and returns 0. Since we have at most $\mu\ell$ sessions and at most μ users, we have:

$$|\Pr [G_1 \text{ returns } 1] - \Pr [G_2 \text{ returns } 1]| = \mu(\ell + 1) \text{Adv}_{\text{Schnorr}}^{\text{ext}}(\lambda).$$

Game G_3 : G_3 is the same as G_2 except that the challenger picks the tuple $(s_*, i_*, j_*, R_*, b_*) \xleftarrow{\$} [\ell] \times [\mu]^2 \times [q] \times \{0, 1\}$ where q is the number of queries to the RO, and returns 0 if E does not occur on the first time on the tuple (s_*, i_*, j_*, R_*) on the last element in the query if $b_* = 1$, otherwise on the second-to-last element if $b_* = 0$. We have:

$$\Pr [G_2 \text{ returns } 1] \leq 2\ell\mu^2 q \Pr [G_3 \text{ returns } 1].$$

We will show that $\Pr [G_3 \text{ returns } 1] \leq \text{Adv}^{\text{CDH}}(\lambda)$

Let $\mathcal{A} \in \text{Poly}(\lambda)$ be an adversary against G_3 . We build the following algorithm $\mathcal{B} \in \text{Poly}(\lambda)$ against CDH. $\mathcal{B}(\mathcal{A}, \mathcal{B})$ simulates G_3 to $\mathcal{A}$, except that:

- If at some stage $\text{Pid}_{i_*}^{s_*} \neq \perp$ and $\text{Pid}_{i_*}^{s_*} \neq j_*$, $\mathcal{B}$ aborts the game (G_1 will return 0 anyway).
- $\mathcal{B}$ sets $\text{pk}_{j_*} \leftarrow B$, and sets $\text{epk}_{i_*}^{s_*} \leftarrow A$, where $\text{epk}_{i_*}^{s_*}$ is the ephemeral public key used in $\pi_{i_*}^{s_*}$, noted X in the protocol (resp. Y) for the initiator (resp. responder). Note that $\mathcal{B}$ does not know the respective discrete logarithms sk_{j_*} and $\text{esk}_{i_*}^{s_*}$ of pk_{j_*} and $\text{epk}_{i_*}^{s_*}$. It is therefore necessary to specify how $\mathcal{B}$ simulates the oracles that would use these values, i.e. oracles $\pi_{j_*}^s$ for all s for sk_{j_*} and the oracle $\pi_{i_*}^{s_*}$ for $\text{esk}_{i_*}^{s_*}$. These two cases are detailed in the two following items.
- **For simulating the oracles $\pi_{j_*}^s$ for all s ,** each time that $\mathcal{B}$ should compute $(\text{epk}_{i_*}^{s_*})^{\text{sk}_{j_*}}$ (i.e. when $\overline{\text{epk}_{j_*}^{s_*}} = \text{epk}_{i_*}^{s_*}$), it sets the tuple $T \leftarrow (\text{epk}_{i_*}^{s_*}, \text{pk}_{j_*})$. If, afterward, $\mathcal{B}$ needs to compute the hash of an element containing $(\text{epk}_{i_*}^{s_*})^{\text{sk}_{j_*}}$

using the random oracle, it replaces it with T and labels the query as '*incomplete*' when storing it in the list of random oracle queries (but still returns a random value corresponding to the query). Each time that $\mathcal{B}$ should compute $(\overline{\text{epk}}_{j_*}^s)^{\text{sk}_{j_*}}$ where $\overline{\text{epk}}_{j_*}^s \neq \text{epk}_{i_*}^{s_*}$, it computes $(\text{pk}_{j_*})^{\overline{\text{esk}}_{j_*}^s}$ instead, where $\overline{\text{esk}}_{j_*}^s$ is the discrete logarithm of $\overline{\text{epk}}_{j_*}^s$. Remember that as long as the proof $\overline{\text{sok}}_{j_*}^s$ is valid, either $\mathcal{B}$ generated $\overline{\text{esk}}_{j_*}^s$ and $\overline{\text{epk}}_{j_*}^s$, or $\overline{\text{epk}}_{j_*}^s$ has been extracted from $\overline{\text{sok}}_{j_*}^s$.

- **For simulating the oracles** $\pi_{i_*}^{s_*}$, if there exists $(i, s) \neq (i_*, s_*)$ such that $\text{epk}_i^s = \overline{\text{epk}}_{i_*}^{s_*}$, then $\mathcal{B}$ generated epk_i^s and knows the secret key esk_i^s , else $\mathcal{B}$ can extract the discrete logarithm $\overline{\text{esk}}_{i_*}^{s_*}$ of $\overline{\text{epk}}_{i_*}^{s_*}$ from $\overline{\text{sok}}_{i_*}^{s_*}$. Note that, in the particular case where $\text{epk}_{i_*}^{s_*} = \overline{\text{epk}}_{i_*}^{s_*}$, while the SOK is valid, it must have been made by the adversary, because it signs the role of the user in the session, so it is not possible to reuse the one simulated by $\mathcal{B}$. Each time that $\mathcal{B}$ should compute $(\overline{\text{epk}}_{i_*}^{s_*})^{\text{esk}_{i_*}^{s_*}}$, it computes $(\text{epk}_{i_*}^{s_*})^{\overline{\text{esk}}_{i_*}^{s_*}}$ instead. Each time that $\mathcal{B}$ should compute $\text{pk}_{j_*}^{\text{esk}_{i_*}^{s_*}}$, it sets the tuple $T \leftarrow (\text{epk}_{i_*}^{s_*}, \text{pk}_{j_*})$. If, afterward, $\mathcal{B}$ needs to compute the hash of an element containing $\text{pk}_{j_*}^{\text{esk}_{i_*}^{s_*}}$ using the random oracle, it replaces $\text{pk}_{j_*}^{\text{esk}_{i_*}^{s_*}}$ by T , and labels the query as '*incomplete*' when storing it in the list of random oracle queries (but still returns a random value corresponding to the query). We note that in the two cases, $T = (A, B)$.
- As opposed to the proofs of the previous protocol $\mathcal{B}$ processes each new query to the random oracle from the adversary that matches the pattern of a previous query labeled '*incomplete*' as other queries. Remember that the advantage of $\mathcal{A}$ drops to 0 if it sends a query that matches an incomplete query before the R_*^{th} query.
- If $\mathcal{B}$ must process the query j_* to RevLTK , $\mathcal{B}$ aborts the game ($\mathcal{A}$ can no longer win G_3 because P_{j_*} must not be corrupted for event E to occur on j_*).
- At the R_*^{th} query to the random oracle, if $b_* = 1$ then $\mathcal{B}$ returns the last element in the query, else it returns the second-to-last element. $\mathcal{B}$ aborts the game.

In this way, G_3 is perfectly simulated for $\mathcal{A}$ except for abortions when $\mathcal{A}$ can no longer win the game or when $\mathcal{B}$ is guaranteed to win their CDH experiment, and therefore:

$$\Pr[G_3 \text{ returns } 1] \leq \text{Adv}^{\text{CDH}}(\lambda).$$

Finally:

$$\Pr\left[\text{Exp}_{\Pi_{\text{CDH}}, \mathcal{A}}^{\text{MITM-privacy}}(\lambda) = 1 \wedge E\right] \leq \mu(\ell + 1)\text{Adv}_{\text{Schnorr}}^{\text{ext}}(\lambda) + 2\ell\mu^2q\text{Adv}^{\text{CDH}}(\lambda),$$

which concludes the proof.

Proof (Theorem 5). This proof is very similar to the one of Theorem 1, the only difference is that Lemma 3 is used instead of Lemma 1 for game G_1 . The final

advantage is then:

$$\left| \Pr \left[\text{Exp}_{\Pi_{\text{CDH}}, \mathcal{A}}^{\text{MITM-privacy}}(\lambda) = 1 \right] - 1/2 \right| \leq \mu(\ell + 1) \text{Adv}_{\text{Schnorr}}^{\text{ext}}(\lambda) + 2\ell\mu^2 q \text{Adv}^{\text{CDH}}(\lambda) + ((\mu\ell)^2 + q^2 + q_r(q + q_r))/2^\lambda + \text{Adv}_{\text{SE}}^{\text{AE}}(\lambda).$$

C.2 Proof of forward-privacy for Π_{CDH}

Theorem 6. *Our protocol Π_{CDH} instantiated with a secure AE scheme and the Schnoor SOK is forward-private under the CDH assumption in the ROM.*

Proof (Theorem 6). We use the following sequence of games.

Game G_0 : This is the experiment $\text{Exp}_{\Pi_{\text{CDH}}, \mathcal{A}}^{\text{Forward-privacy}}(\lambda)$. We have that:

$$\Pr \left[\text{Exp}_{\Pi_{\text{CDH}}, \mathcal{A}}^{\text{Forward-privacy}}(\lambda) = 1 \right] = \Pr[G_0 \text{ returns } 1].$$

Game G_1 : G_1 is the same as G_0 except that the challenger picks $(i_*, j_*, k_*, r_*) \xleftarrow{\$} [\mu]^3 \times [\ell]$, and if $i|j \neq i_*|j_*$ or if $\pi_{k_*}^{r_*}$ is not the partner oracle of $\pi_{i_*|j_*}^1$, then G_1 aborts and returns a random bit (note that according to the winning condition of the experiment, the returned oracle $\pi_{i|j}^1$ must have a partner oracle at the end of the game). We have:

$$\Pr[G_0 \text{ returns } 1] \leq \ell\mu^3 \Pr[G_1 \text{ returns } 1].$$

Game G_2 : G_2 is the same as G_1 except that if the RO returns the same random element for two distinct queries, then G_2 sets $\text{abort}_2 = \text{true}$, aborts and returns a random bit. We set q as the number of queries to the random oracle during G_2 , we have:

$$|\Pr[G_2 \text{ returns } 1] - \Pr[G_1 \text{ returns } 1]| \leq \Pr[\text{abort}_2] \leq q^2/|\mathcal{K}|.$$

Game G_3 : G_3 is the same as G_2 except that the challenger replaces all the signatures of knowledge it has to generate by simulations. Since the Schnorr signature of knowledge is perfectly zero-knowledge, we have:

$$\Pr[G_2 \text{ returns } 1] = \Pr[G_3 \text{ returns } 1].$$

Game G_4 : G_4 is the same as G_3 except that the challenger picks $(R_*, b_*) \xleftarrow{\$} [q] \times \{0, 1\}$, and if the adversary sends a query to the random oracle ending with three concatenated group elements such that the second or third element from the end is $g^{\text{esk}_{i_*|j_*}^1 \text{esk}_{k_*}^{r_*}}$, then the challenger aborts and returns a random bit if:

- this not appends for the first time at the R_*^{th} query, or
- when it appends for the first time, $g^{\text{esk}_{i_*|j_*}^1 \text{esk}_{k_*}^{r_*}}$ is the second (resp thrid) element from the end if $b_* = 0$ (resp. $b_* = 1$).

We have:

$$\Pr[G_3 \text{ returns } 1] \leq 2q\Pr[G_4 \text{ returns } 1].$$

Game G_5 : G_5 is the same as G_4 except that if the adversary sends a query ending with three concatenated group elements such that the second or third element from the end is $g^{\text{esk}_{i_*|j_*}^1 \text{esk}_{k_*}^{r_*}}$ to the random oracle, then G_5 sets $\text{abort}_5 = \text{true}$, aborts and returns a random bit. We claim that:

$$|\Pr[G_4 \text{ returns } 1] - \Pr[G_5 \text{ returns } 1]| \leq \Pr[\text{abort}_5] \leq \text{Adv}^{\text{CDH}}(\lambda)$$

We prove the claim by reduction. Let $\mathcal{A} \in \text{Poly}(\lambda)$ be an adversary such that $\Pr[\text{abort}_5]$ is non-negligible. We build the following algorithm $\mathcal{B} \in \text{Poly}(\lambda)$ against CDH. $\mathcal{B}(A, B)$ simulates G_5 to $\mathcal{A}$, except that:

- $\mathcal{B}$ sets $\text{epk}_{i_*|j_*}^1 \leftarrow A$ and $\text{epk}_{k_*}^{r_*} \leftarrow B$, where $\text{epk}_{i_*|j_*}^1$ and $\text{epk}_{k_*}^{r_*}$ are the ephemeral public keys used in $\pi_{i_*|j_*}^1$ and $\pi_{k_*}^{r_*}$, noted in the protocol X (resp. Y) for the initiator (resp. responder).
- Recall that $\mathcal{A}$ must ensure that $\pi_{i_*|j_*}^1$ and $\pi_{k_*}^{r_*}$ are partners (and so $\text{Pid}_{i_*|j_*}^1 = k_*$) in order to maintain a non-negligible advantage according to G_1 . As soon as this condition can no longer be satisfied, $\mathcal{B}$ aborts the game by returning a random bit. When it simulates $\pi_{i_*|j_*}^1$, $\mathcal{B}$ computes $(\text{epk}_{i_*|j_*}^1)^{\text{sk}_{k_*}}$ instead of $\text{pk}_{k_*}^{\text{esk}_{i_*|j_*}^1}$. Moreover, when $\mathcal{B}$ should compute $(\text{epk}_{k_*}^{r_*})^{\text{esk}_{i_*|j_*}^1}$, it replaces it with $T = (A, B)$. It then labels the queries to the random oracle containing T as 'incomplete' when storing it in the list of random oracle queries (but still returns a random value corresponding to the query).
- In a similar way, when it simulates $\pi_{k_*}^{r_*}$, $\mathcal{B}$ computes $(\text{epk}_{k_*}^{r_*})^{\text{sk}_{i_*|j_*}^1}$ instead of $\text{pk}_{i_*|j_*}^{\text{esk}_{k_*}^{r_*}}$. Moreover, when $\mathcal{B}$ should compute $(\text{epk}_{i_*|j_*}^1)^{\text{esk}_{k_*}^{r_*}}$, it replaces it with $T = (A, B)$. It then labels the queries to the random oracle containing T as 'incomplete' when storing it in the list of random oracle queries (but still returns a random value corresponding to the query).
- As opposed to the proofs of the previous protocol $\mathcal{B}$ processes each new query to the random oracle from the adversary that matches the pattern of a previous query labeled 'incomplete' as other queries. Remember that the advantage of $\mathcal{A}$ drops to 0 if it sends a query that matches an incomplete query before the R_*^{th} query.
- At the R_*^{th} query to the random oracle, if $b_* = 1$ then $\mathcal{B}$ returns the third element from the end in the query, else it returns the second element from the end. $\mathcal{B}$ then aborts the game.

To keep an advantage in the game, the adversary must not interfere with communication between $\pi_{i_*|j_*}^1$ and $\pi_{k_*}^{r_*}$ (they must be exclusive partner oracles), and therefore cannot modify the ephemeral keys exchanged. In this way, G_5 is perfectly simulated at $\mathcal{A}$ except for abortions when $\mathcal{A}$ can no longer win the game or when $\mathcal{B}$ is guaranteed to win their CDH experiment, and therefore $\Pr[\text{abort}_5] \leq \text{Adv}^{\text{CDH}}(\lambda)$, which concludes the proof of the claim.

From this step, the abort conditions in games 5 and 2 ensures that while the adversary keeps an advantage, the keys generated by the session seems to their fully random, and are never reused in another session (except the key shared with $\pi_{k_*}^{r_*}$). As a result, the latest information that the adversary can exploit about the identity of $P_{i|j}$ is the message encrypted with the key K , which contains its public key (if $P_{i|j}$ plays the role of initiator).

Game G_6 : G_6 is the same as G_5 except that if the role of $P_{i|j}$ in $\pi_{i|j}^1$ is initiator, then it computes $C \leftarrow \text{Enc}_K(\perp)$ instead of computing $C \leftarrow \text{Enc}_K(U)$ at the second interaction. If during the simulation of an oracle the challenger has to decrypt C with the key K , they do not decrypt C and act as if the decryption returned U . Furthermore, if during the simulation of an oracle the challenger has to decrypt a ciphertext $C' \neq C$ with the key K , they act as if the decryption had failed.

We claim that:

$$|\Pr[G_5 \text{ returns } 1] - \Pr[G_6 \text{ returns } 1]| \leq \text{Adv}_{\text{SE}}^{\text{AE}}(\lambda).$$

This claim can be proved in the same way as for G_4 in the MITM-privacy proof for Π_{SDH} .

Game G_7 : G_7 is the same as G_6 except that at the end of the game, if the oracle RevSessKeyPriv generated a random session key k^* equals to either another session key randomly generated by the same oracle or an output of the random oracle, then G_7 sets $\text{abort}_7 = \text{true}$, aborts and returns a random bit. We set q (resp. q_r) as the number of queries to the random oracle (resp. RevSessKeyPriv) during G_4 , we have:

$$|\Pr[G_6 \text{ returns } 1] - \Pr[G_7 \text{ returns } 1]| \leq \Pr[\text{abort}_7] \leq q_r(q + q_r)/|\mathcal{K}|.$$

For the same reasons as for G_5 in the proof of Theorem 2, we have:

$$\Pr[G_7 \text{ returns } 1] = 1/2.$$

By considering that $|\mathbb{G}| > 2^\lambda$ and $|\mathcal{K}| > 2^\lambda$, we can therefore conclude that:

$$\begin{aligned} & \left| \Pr \left[\text{Exp}_{\Pi_{\text{CDH}}, \mathcal{A}}^{\text{Forward-privacy}}(\lambda) = 1 \right] - 1/2 \right| \leq \\ & 2l\mu^3 q \left(\text{Adv}^{\text{CDH}}(\lambda) + (q + q^2 + q_r(q + q_r))/2^\lambda + \text{Adv}_{\text{SE}}^{\text{AE}}(\lambda) \right), \end{aligned}$$

which concludes the proof.

C.3 Proof of key-secrecy for Π_{CDH}

Theorem 7. *Our protocol Π_{CDH} instantiated with a secure AE scheme and the Schnorr SOK is key-secret under the CDH assumption in the ROM.*

Lemma 4. *Let E be the event: "at the end of the key-secrecy experiment, for some tuple (s, i, j, R) , P_j is never corrupted, $\text{Pid}_i^s = j$, and the R^{th} query to the RO ends with concatenated group elements such that the last or second-to-last element is $g^{\text{esk}_i^s \text{sk}_j}$, where esk_i^s is the ephemeral key used in π_i^s ". For any PPT adversary $\mathcal{A}$ playing the key-secrecy experiment on Π_{CDH} , the probability that E occurs and $\mathcal{A}$ wins the experiment is negligible under the CDH assumption in the ROM.*

Proof. This lemma can be proved in a manner similar to Lemma 3. The only notable difference is that in this Lemma, we do not require that P_i be uncorrupted, which is never used in the proof of Lemma 3.

Proof. We show that Π_{CDH} is key secret using the following sequences of games.

Game G_0 . In this game, the adversary interacts with honestly behaving oracles according to the key-secrecy security experiment. Oracles derive session keys exactly as specified in the protocol. The adversary's goal is to distinguish a real session key from a random one. We define:

$$\Pr[\text{break}_{\text{KE}}] = \Pr[G_0 \text{ returns } 1].$$

Game G_1 . This game is identical to G_0 except that the challenger aborts if at the end of the game, for some tuple (s, i, j, R) , P_j is not corrupted, $\text{Pid}_i^s = j$, and the R^{th} query to the RO ends with concatenated group elements such that the last or second-to-last element is $g^{\text{esk}_i^s \text{sk}_j}$.

By Lemma 4, we have that:

$$|\Pr[G_0 \text{ returns } 1] - \Pr[G_1 \text{ returns } 1]| \leq \mu(\ell + 1)\text{Adv}_{\text{Schnorr}}^{\text{ext}}(\lambda) + 2\ell\mu^2q\text{Adv}^{\text{CDH}}(\lambda).$$

Game G_2 : G_2 is the same as G_1 except that the challenger replaces all the signatures of knowledge it has to generate by simulations. Since the Schnorr signature of knowledge is perfectly zero-knowledge, we have:

$$\Pr[G_1 \text{ returns } 1] = \Pr[G_2 \text{ returns } 1]$$

Game G_3 : G_3 is the same as G_2 except that the challenger runs the knowledge extractor on the fresh signatures of knowledge generated by the adversary and stores the corresponding pair of element/discrete logarithm when simulating the session oracles. If one extraction fails, it aborts the game and returns 0. Since we have at most $\mu\ell$ sessions and at most μ users, we have:

$$|\Pr[G_2 \text{ returns } 1] - \Pr[G_3 \text{ returns } 1]| = \mu(\ell + 1)\text{Adv}_{\text{Schnorr}}^{\text{ext}}(\lambda).$$

Game G_4 . This game is identical to G_3 except that the challenger aborts the experiment if two oracles in the same role (either initiators or responders) generate identical session contexts. This implies a collision in their ephemeral exponents,

which is highly unlikely. Given at most ℓ sessions per user and μ users, this collision occurs with probability at most:

$$|\Pr[G_3 \text{ returns } 1] - \Pr[G_4 \text{ returns } 1]| \leq \frac{(\mu\ell)^2}{|\mathbb{G}|}.$$

Game G_5 . This game is identical to G_4 except that the challenger guesses a tuple (i_*, s_*, j_*, t_*) such that if the adversary $\mathcal{A}$ ever queries the **Test** oracle, it will be on $\pi_{i_*}^{s_*}$ and if that oracle has an origin oracle, it is $\pi_{j_*}^{t_*}$ (Note that t_* is unique, according to G_3). If the adversary $\mathcal{A}$ fails its guess, the game aborts. We have:

$$\Pr[G_4 \text{ returns } 1] \leq \mu^2 \ell^2 \cdot \Pr[G_5 \text{ returns } 1].$$

Game G_6 : G_6 is the same as G_5 except that the challenger picks $(R_*, b_*) \xleftarrow{\$} [q] \times \{0, 1\}$, and if the adversary sends a query to the random oracle ending with three concatenated group elements such that the second or third element from the end is $g^{\text{esk}_{i_*}^{s_*} \text{esk}_{j_*}^{t_*}}$, then the challenger aborts and returns a random bit if:

- this not appends for the first time at the R_*^{th} query, or
- when it appends for the first time, $g^{\text{esk}_{i_*}^{s_*} \text{esk}_{j_*}^{t_*}}$ is the second (resp thrid) element from the end if $b_* = 0$ (resp. $b_* = 1$).

We have:

$$\Pr[G_5 \text{ returns } 1] \leq 2q \Pr[G_6 \text{ returns } 1].$$

Game G_7 . This game is identical to G_6 except that the game sets **abort**₇ = **true**, aborts and returns a random bit if $\pi_{j_*}^{t_*}$ is the origin oracle of $\pi_{i_*}^{s_*}$, $\pi_{i_*}^{s_*}$ is fresh and the adversary has sent a query to the random oracle containing an element equal to $g^{\text{esk}_{i_*}^{s_*} \text{esk}_{j_*}^{t_*}}$ during the game. We claim that :

$$|\Pr[G_6 \text{ returns } 1] - \Pr[G_7 \text{ returns } 1]| \leq \Pr[\text{abort}_7] \leq \text{Adv}^{\text{CDH}}(\lambda).$$

We prove this claim by reduction. Let $\mathcal{A} \in \text{Poly}(\lambda)$ be an adversary such that $\Pr[\text{abort}_7]$ is non-negligible. We build the following algorithm $\mathcal{B} \in \text{Poly}(\lambda)$ against CDH $\mathcal{B}(A, B)$ simulates G_7 to $\mathcal{A}$, except that:

- $\mathcal{B}$ sets $\text{epk}_{j_*}^{t_*} \leftarrow B$, and sets $\text{epk}_{i_*}^{s_*} \leftarrow A$, where $\text{epk}_{i_*}^{s_*}$ and $\text{epk}_{j_*}^{t_*}$ are the ephemeral public keys, respectively, used in $\pi_{i_*}^{s_*}$ and $\pi_{j_*}^{t_*}$. If $\overline{\text{epk}_{i_*}^{s_*}} \neq \text{epk}_{j_*}^{t_*}$, then $\mathcal{B}$ aborts the game (in this case $\pi_{j_*}^{t_*}$ is not the origin oracle of $\pi_{i_*}^{s_*}$). Note that $\mathcal{B}$ does not know the respective discrete logarithms $\text{esk}_{j_*}^{t_*}$ and $\text{esk}_{i_*}^{s_*}$ of $\text{epk}_{j_*}^{t_*}$ and $\text{epk}_{i_*}^{s_*}$. It is therefore necessary to specify how $\mathcal{B}$ simulates the oracles that would use these values, i.e. oracle $\pi_{j_*}^{t_*}$ for $\text{esk}_{j_*}^{t_*}$ and the oracle $\pi_{i_*}^{s_*}$ for $\text{esk}_{i_*}^{s_*}$.
- In $\pi_{i_*}^{s_*}$, when $\mathcal{B}$ should compute $(\text{epk}_{j_*}^{t_*})^{\text{esk}_{i_*}^{s_*}}$, it sets a tuple $T \leftarrow (A, B)$. If, afterward, $\mathcal{B}$ needs to compute the hash of an element containing $(\text{epk}_{j_*}^{t_*})^{\text{esk}_{i_*}^{s_*}}$ using the random oracle, it replaces it with T and labels the query as ‘*incomplete*’ when storing it in the list of random oracle queries (but still returns a random value corresponding to the query).

- In $\pi_{i_*}^{s_*}$, when $\mathcal{B}$ should compute $(\text{pk}_{\text{Pid}_{i_*}^{s_*}})^{\text{esk}_{i_*}^{s_*}}$, it computes $(\text{epk}_{i_*}^{s_*})^{\text{sk}_{\text{Pid}_{i_*}^{s_*}}^{s_*}}$ instead. In the case where $\mathcal{B}$ does not know the secret key $\text{sk}_{\text{Pid}_{i_*}^{s_*}}^{s_*}$ (i.e. $\text{pk}_{\text{Pid}_{i_*}^{s_*}}^{s_*}$ has been chosen by $\mathcal{A}$), then $\mathcal{B}$ aborts the game. Recall that if $\pi_{i_*}^{s_*}$ is fresh and has an origin oracle, then $P_{\text{Pid}_{i_*}^{s_*}}$ was not corrupted before $\pi_{i_*}^{s_*}$ accepts, so $\mathcal{B}$ knows the secret key of $P_{\text{Pid}_{i_*}^{s_*}}$.
- In $\pi_{j_*}^{t_*}$, when $\mathcal{B}$ should compute $(\text{epk}_{i_*}^{s_*})^{\text{esk}_{j_*}^{t_*}}$, it sets a tuple $T \leftarrow (A, B)$. If, afterward, $\mathcal{B}$ needs to compute the hash of an element containing $(\text{epk}_{i_*}^{s_*})^{\text{esk}_{j_*}^{t_*}}$ using the random oracle, it replaces it with T and labels the query as ‘incomplete’ when storing it in the list of random oracle queries (but still returns a random value corresponding to the query).
- In $\pi_{j_*}^{t_*}$, if $\text{epk}_{i_*}^{s_*} \neq \overline{\text{epk}}_{j_*}^{t_*}$, then when $\mathcal{B}$ should compute $(\overline{\text{epk}}_{j_*}^{t_*})^{\text{esk}_{j_*}^{t_*}}$, it computes $(\text{epk}_{j_*}^{t_*})^{\text{esk}_{j_*}^{t_*}}$ instead, where $\overline{\text{esk}}_{j_*}^{t_*}$ denotes the discrete logarithm of $\overline{\text{epk}}_{j_*}^{t_*}$. We emphasize that $\mathcal{B}$ still knows $\overline{\text{esk}}_{j_*}^{t_*}$: either it generated $\overline{\text{epk}}_{j_*}^{t_*}$ earlier in the game and $\mathcal{A}$ reuses it, or $\overline{\text{epk}}_{j_*}^{t_*}$ was chosen by the adversary, in which case the adversary had to provide a signature of knowledge $\overline{\text{sok}}_{j_*}^{t_*}$ from which $\mathcal{B}$ was able to extract $\overline{\text{esk}}_{j_*}^{t_*}$. Note that the adversary cannot reuse the (simulated) signatures of knowledge corresponding to $\text{epk}_{j_*}^{t_*}$, since the signed role does not match the session ($\pi_{j_*}^{t_*}$ aborts if it receives a signature of its own role).
- In $\pi_{j_*}^{t_*}$, when $\mathcal{B}$ should compute $(\text{pk}_{\text{Pid}_{j_*}^{t_*}})^{\text{esk}_{j_*}^{t_*}}$, it computes $(\text{epk}_{j_*}^{t_*})^{\text{sk}_{\text{Pid}_{j_*}^{t_*}}^{t_*}}$ instead, where $\text{sk}_{\text{Pid}_{j_*}^{t_*}}$ denotes the discrete logarithm of $\text{pk}_{\text{Pid}_{j_*}^{t_*}}$. We emphasize that $\mathcal{B}$ still knows $\text{sk}_{\text{Pid}_{j_*}^{t_*}}$: either it chooses it, or $\text{pk}_{\text{Pid}_{j_*}^{t_*}}$ was chosen by the adversary at the time of the registration of $P_{\text{Pid}_{j_*}^{t_*}}$, in which case the adversary had to provide a signature of knowledge $\text{sok}_{\text{Pid}_{j_*}^{t_*}}$ from which $\mathcal{B}$ was able to extract $\text{sk}_{\text{Pid}_{j_*}^{t_*}}$. Note that the adversary cannot reuse the (simulated) signatures of knowledge corresponding to $\text{epk}_{j_*}^{t_*}$ and $\text{epk}_{i_*}^{s_*}$, since the message signed by $\text{sok}_{\text{Pid}_{j_*}^{t_*}}$ must be $P_{\text{Pid}_{j_*}^{t_*}}$, and not a bit corresponding to a role.
- As opposed to the proofs of the previous protocol $\mathcal{B}$ processes each new query to the random oracle from the adversary that matches the pattern of a previous query labeled ‘incomplete’ as other queries. Remember that the advantage of $\mathcal{A}$ drops to 0 if it sends a query that matches an incomplete query before the R_*^{th} query.
- At the R_*^{th} query to the random oracle, if $b_* = 1$ then $\mathcal{B}$ returns the third element from the end in the query, else it returns the second element from the end. $\mathcal{B}$ then aborts the game.

In this way, G_7 is perfectly simulated at $\mathcal{A}$ except for abortions when $\mathcal{A}$ can no longer win the game or when $\mathcal{B}$ is guaranteed to win their CDH experiment, and therefore $\Pr[\text{abort}_7] \leq \text{Adv}^{\text{CDH}}(\lambda)$,

Game G_8 . This game is identical to G_7 except that the challenger aborts the game if the RO outputs two times the same value on two different queries, or outputs k_1 . We set q as the number of queries to the RO, we have:

$$|\Pr[G_8 \text{ returns } 1] - \Pr[G_7 \text{ returns } 1]| \leq \Pr[\text{abort}_8] \leq (q+1)^2/|\mathcal{K}|.$$

Remember that in order to have a significant probability of winning, the tested oracle ($\pi_{i_*}^{s*}$ according to G_5) must remain fresh, which means that the session keys of the tested oracle's partner oracles must not have been revealed. Note that given the G_4 game, since session keys are always different, the tested oracle can only have one partner, and given the G_8 game, two oracles that are not partners have a different context and necessarily have different session keys. As a result, $\mathcal{A}$ cannot deduce the tested session key by revealing the session keys of other oracles. Finally, for the adversary to maintain a significant advantage, they must be in one of these two situations required by freshness of the tested session $\pi_{i_*}^{s*}$:

- $P_{\text{Pid}_{i_*}^{s*}}$ was not corrupted before $\pi_{i_*}^{s*}$ accepted if $\pi_{i_*}^{s*}$ has an origin-oracle. In this case, the game G_7 ensures that the adversary must not have sent the key $(\text{epk}_{i_*}^{s*})^{\text{esk}_{i_*}^{s*}}$ in a query to the RO, which is necessary to generate the oracle's session key.
- $P_{\text{Pid}_{i_*}^{s*}}$ was not corrupted at all if $\pi_{i_*}^{s*}$ has no origin-oracle. In this case, the game G_1 ensures that the adversary must not have sent the key $(\text{pk}_{\text{Pid}_{i_*}^{s*}})^{\text{esk}_{i_*}^{s*}}$ in a query to the RO, which is necessary to generate the oracle's session key.

In both cases, the session key of $\pi_{i_*}^{s*}$ is therefore perfectly random from the point of view of $\mathcal{A}$. It therefore has no way of guessing whether the key returned by the test oracle is the real session key or a randomly chosen key. We deduce that:

$$\Pr[G_8 \text{ returns } 1] = 1/2.$$

Combining all bounds, and by considering that $|\mathbb{G}| > 2^\lambda$ and $|\mathcal{K}| > 2^\lambda$, we obtain:

$$\Pr[\text{break}_{\text{KE}}] \leq 2\mu(\ell+1)\text{Adv}_{\text{Schnorr}}^{\text{ext}}(\lambda) + \frac{(\mu\ell)^2}{2^\lambda} + 2q(\mu\ell)^2 \left(2\text{Adv}^{\text{CDH}}(\lambda) + \frac{(q+1)^2}{2^\lambda} \right).$$

Furthermore, the probability of having two partner oracles at the same oracle is bounded by the probability that two oracles generate the same ephemeral key, and therefore:

$$\Pr[\text{break}_{\text{Unique}}] \leq \frac{(\mu\ell)^2}{|\mathbb{G}|}.$$

Finally, let us consider two partner oracles π_i^s and π_j^t , i.e., which share the same context and transcript, and satisfy $\text{Pid}_i^s = j$ and $\text{Pid}_j^t = i$. We will have:

$$\begin{aligned} k_i^s &= H(\text{ctxt} \| (\text{pk}_j)^{\text{esk}_i^s} \| (\text{epk}_j^t)^{\text{esk}_i^s} \| (\text{epk}_j^t)^{\text{sk}_i}) = H(\text{ctxt} \| g^{\text{sk}_j \text{esk}_i^s} \| g^{\text{esk}_j^t \text{esk}_i^s} \| g^{\text{esk}_j^t \text{sk}_i}); \\ k_j^t &= H(\text{ctxt} \| (\text{epk}_i^s)^{\text{sk}_j} \| (\text{epk}_i^s)^{\text{esk}_j^t} \| (\text{pk}_i)^{\text{esk}_j^t}) = H(\text{ctxt} \| g^{\text{sk}_j \text{esk}_i^s} \| g^{\text{esk}_j^t \text{esk}_i^s} \| g^{\text{esk}_j^t \text{sk}_i}). \end{aligned}$$

We deduce that $k_j^t = k_i^s$, then:

$$\Pr[\text{breakSound}] \leq \Pr[k_i^s \neq k_j^t] = 0.$$

To conclude, we obtain the following bound for the AKE advantage:

$$\text{Adv}_{\text{PPAKE}, \mathcal{A}}^{\text{AKE}}(\lambda) \leq 2\mu(\ell+1)\text{Adv}_{\text{Schnorr}}^{\text{ext}}(\lambda) + \frac{(\mu\ell)^2}{2^\lambda} + 2q(\mu\ell)^2 \left(2\text{Adv}^{\text{CDH}}(\lambda) + \frac{(q+1)^2}{2^\lambda} \right).$$